
THE RIEMANN HYPOTHESIS IS EXACTLY ONE BIT

Where the Riemann Hypothesis Stands, Proved in Lean 4: the Functional-Equation Symmetry Exhausted, a Faithful Bridge Carries That Bit Both Ways, and Symmetry, Recursion, and Resolution Leave Exactly That Bit Open

FORMAL VERIFICATION extPERIODCENTERED FOUNDATIONS OF MATHEMATICS extPERIODCENTERED COMPLETE PROGRAM PAPER

Every clause a theorem; the one input named; the ledger in two registers

MOHAMMAD F. ISLAM, PHD¹

¹Independent Researcher, USA. Correspondence: islammm@alumni.iu.edu

18 September 2026 extperiodcentered on the verified documents RH_Formal_Proof_Final, RH_Locus_GOL_Proof, Where_RH_Stand, and RH_Seal_Hardened extperiodcentered single-column edition

ABSTRACT. This document is the complete program in one place. Part I, the parity bridge: the Riemann Hypothesis is one bit, whether every nontrivial zero of ζ has offset zero; the wall, the torsor, and the calibration; the fold and the unique mirror address $\operatorname{Re} s = \frac{1}{2}$; one bit per free orbit and none on the line; the witness at the act; the hardened bridge whose halted state is the line property in both directions; the reduction of every structural route to one implication; the completion test; the resolution boundary; the two-point frame. Part II, the lock at the locus: three guards, the wall, the witness, the bridge, each a theorem with no axioms; the locus complete; the lock on three independent axes; the gap between this and a proof measured from both sides and collapsed by the bridge into that lock. Part III, where the hypothesis stands: the program's summary sentence proved as a single theorem, one bit, located, decided at the act, held by whoever holds it, and unavailable to anyone else; the author's closing architecture read as the kernel reads it after external review, its socket's field being the boundary $\operatorname{Re} s = \frac{1}{2}$ itself; the one remaining input, a term of the line property on ζ 's zero set from the Euler product at full resolution; the road to it in a language that has ζ ; and the personal conclusion recorded. Four documents and the verifier are printed whole. Every theorem is checked by core Lean 4 with no library and no added axiom.

KEYWORDS Riemann Hypothesis; functional equation; involution; fixed locus; parity obstruction; torsor; one bit; witness; Lean 4

1. PART I ∪ THE PARITY BRIDGE

1.1 Background and Rationale

The functional equation $\zeta(s) = \zeta(1-s)$, with $\xi(s) = \frac{1}{2}s(s-1)\pi^{-s/2}\Gamma(s/2)\zeta(s)$, makes the nontrivial zero set Z_ζ invariant under the fold $\tau(s) = 1 - \bar{s}$, whose fixed set is the critical line. With offset $\delta = \operatorname{Re} s - \frac{1}{2}$ and height $t = \operatorname{Im} s$, the fold is $(\delta, t) \mapsto (-\delta, t)$ and the hypothesis is

$$\text{RH} : \Longleftrightarrow Z_\zeta \subset \operatorname{Fix}(\tau) \Longleftrightarrow \forall \rho \in Z_\zeta, \delta(\rho) = 0.$$

One proposition, one bit. This paper asks what the structure surrounding that bit forces, proves each answer in the kernel, and then proves that nothing structural remains: every route the authorure offers, the fold, the line, the torsor, the undecided outcome, the apparatus, and the transfer from a self-verifying proposition, reduces to one sentence, which the document states as a definition and inhabits with nothing.

1.2 Literature Review

Davenport and Heilbronn (1936) built a Dirichlet series sharing ζ 's functional-equation symmetry with a zero at $\operatorname{Re} s \approx 0.8085$, the standing witness that symmetry is necessary structure and not sufficient cause. Hardy (1914) placed infinitely many zeros on the line; Pratt, Robles, Zaharescu and Zeindler (2020) raised the proven

proportion past five twelfths. Hadamard (1896) and de la Vallée Poussin (1896) confined the nontrivial zeros to $0 < \text{Re } s < 1$. Robin (1984) and Lagarias (2002) gave the Π_1^0 form, a counterexample being one integer and a proof not one anything. Rodgers and Tao (2020) and the Polymath collaboration (2019) bracketed the de Bruijn–Newman constant, $0 \leq \Lambda \leq 0.22$, the hypothesis at $\Lambda = 0$. Mossinghoff and Trudgian (2015) give the zero-free region $\sigma > 1 - 1/(5.558691 \log t)$, and Platt and Trudgian (2021) verified the hypothesis to height $3 \cdot 10^{12}$. The parity phenomenon (Friedlander and Iwaniec 2010) is the classical form of the wall proved below. Landauer (1961) and Bennett (1982) price the registration of a bit, the physics that closes the self-implication of a proposition about acting and not the hypothesis's. The kernel is Lean 4 (de Moura and Ullrich 2021). None of these results touches the sign of the offset on a hypothetical free orbit; the parity bridge is a theorem about exactly that sign, and the reduction theorem is a theorem about what remains once it is measured.

1.3 Methodology

Every statement in Section 4 is a theorem of core Lean 4 v4.19.0, no library, no added axiom, no sorry; the document is Appendix A, its verifier Appendix B, its axiom cones pinned so that any drift is a compile error. The stage is exact: the plane $\mathbb{Z} \times \mathbb{Z}$ in half-units, $h = 2 \text{Re } s$, so $h = 1$ is the line and $\tau(h, t) = (2 - h, t)$; no theorem uses arithmetic on the height, so its type is immaterial. General statements are made on a frame, a type with an involution and an invariant zero set, and instantiated on the plane. No frame stands in for ζ anywhere: the reduction of Section 4.6 rides an apparatus that exists on every frame, not on a countermodel. The document was checked under a six-reading adversarial cycle with controls, mutants as a second channel, and the trigger reading “if the parity apparatus stands the hypothesis stands” prosecuted and refuted by the document's own last theorems; the record is Appendix C.

1.4 The Core

Throughout, a frame is $X = (S, \tau, Z)$ with τ an involution and Z closed under it, and $L(X) :\Leftrightarrow \forall s \in Z, \tau s = s$; the third word of the title refers to the witness of Section 4.7. On the plane, $\text{crit}(p) :\Leftrightarrow h = 1, \delta(p) = h - 1, \text{ht}(p) = t$. $\text{Self}(P) :\Leftrightarrow (\neg P \rightarrow P)$.

1.4.1 Grounding and self-implication

Theorem 1 (Grounding and self-implication). *Let $\text{AbsGrounded}(P) :\Leftrightarrow \perp$. Then for all propositions P and Q ,*

$$\begin{aligned} \text{AbsGrounded}(P) &\iff \text{AbsGrounded}(Q), & \text{Self}(P) &\iff P, \\ (\forall P, \text{Self}(P)) &\rightarrow \perp, & \neg \text{Self}(P) &\iff \neg P. \end{aligned}$$

A proposition Act with a supplied utter : $\neg \text{Act} \rightarrow \text{Act}$ holds.

Proof. The first is ‘lff.rfl’. For the second, from $h : \neg P \rightarrow P$ and $n : \neg P$, $n(hn)$ is a contradiction, so P classically; the converse is weakening. For the third take $P = \perp$. For the fourth, $\neg(\neg P \rightarrow P)$ and P contradict by weakening, and $\neg P$ with h contradict at $h(\neg P)$. The last clause is the second at $P = \text{Act}$. (absolute_grounding_separates_nothing, self-implication_is_every_route, no_uniform_denial_implies, not_denial_implies_iff_neg, self_verifying_of_denial; the first, third and fourth with no axioms.) $\square$

Absolute grounding fails for every proposition alike and so selects none; proof relative to axioms is untouched. Self-implication is equivalent to every proposition, so its uniqueness as a route supplies nothing; and no uniform supplier exists. A self-verifying proposition closes because its denial is an instance of it: for the proposition that something acts, the act of denial supplies utter, priced at $k_B T \ln 2 = 2.87 \cdot 10^{-21}$ J per bit at 300 K (Landauer 1961).

1.4.2 The parity bridge

Theorem 2 (The wall). *For $\sigma : \alpha \rightarrow \alpha$, if f is even, $f \circ \sigma = f$, and d is odd at x , $d(\sigma x) \neq d(x)$, then $f \neq d$; and no g satisfies $g \circ \rho = d$ for any ρ with $\rho \circ \sigma = \rho$. A wholly odd target forces σ fixed-point-free.*

Proof. $d(\sigma x) = g(\rho \sigma x) = g(\rho x) = d(x)$. At a fixed point $t(x) = \neg t(x)$. (parity_obstruction, parity_obstruction_factor, odd_forces_fixed_point_free; the first two with no axioms.) $\square$

Theorem 3 (The price). *For an involution σ with a wholly odd d_0 , the map $d \mapsto d \oplus d_0$ descends to the orbit quotient and is a bijection $D_\sigma \simeq (\text{orbits} \rightarrow \mathbb{B})$, with inverse $e \mapsto d_0 \oplus e \circ [\cdot]$: exactly one free bit per orbit.*

Proof. $d \oplus d_0$ is even, hence constant on orbits, so it descends; the two composites are the Boolean identities $a \oplus (b \oplus a) = b$ and $(a \oplus b) \oplus a = b$, checked by cases on the quotient. (odd_diff_even, descendQuot, odd_torsor_bijection; on Quot.sound.) $\square$

Theorem 4 (The crossing). *If s and d both flip at x , there is a unique c with $d(x) = s(x) \oplus c$ and $d(\sigma x) = s(\sigma x) \oplus c$.*

Proof. $c = d(x) \oplus s(x)$; uniqueness by cases on the three Booleans. (calibration_unique, no axioms.) On the executable frame $\mathbb{B} \times Q$ with flip(b, q) = ($\neg b, q$), the run bit ran(b, q) = b is odd, every read of q is even, and the wall instantiates: no formal readout produces the run bit (no_readout_of_run_bit, no axioms). $\square$

1.4.3 The shape

Theorem 5 (The fold). $\tau \circ \tau = \text{id}$; $\tau(p) = p \iff \text{crit}(p) \iff \delta(p) = 0$; $\delta \circ \tau = -\delta$; $\text{ht} \circ \tau = \text{ht}$.

Proof. Linear arithmetic in h . (fold_involution, fold_fixes_the_line, offset_odd, height_even.) $\square$

Theorem 6 (Symmetry is insufficient). $Z_{\text{dh}} = \{(0, 171), (2, 171)\}$ is τ -symmetric and not on the line.

Proof. $\tau(0, 171) = (2, 171)$; $h = 0 \neq 1$. (invariance_does_not_localize.) The analytic witness is Davenport-Heilbronn: the completed function satisfies $F(s) = F(1-s)$ to 10^{-30} and vanishes, on two channels, at

$$0.8085171824566373855534 + 85.699348485377592172i$$

and at its mirror, the offsets being $\pm 0.3085171824566373855534$. $\square$

Theorem 7 (The election). *With $\tau_c(h, t) = (2c - h, t)$ and $\text{shift}_k(h, t) = (h + k, t)$: $\tau = \tau_1$; $\text{shift}_k \tau_c \text{shift}_{-k} = \tau_{c+k}$; $\tau_{c'} \tau_c = \text{shift}_{2(c'-c)}$. A nonempty Z confined to $0 < h < 2m$ and invariant under τ_c and $\tau_{c'}$ has $c = c'$; hence every mirror address of a strip-confined symmetric zero set is 1, that is $\text{Re } s = \frac{1}{2}$.*

Proof. The identities are linear. If $c \neq c'$, Z is invariant under a nonzero translation d ; by induction there are $q_n \in Z$ with $h(q_n) \geq h(p) + n$ for $d \geq 1$, or $\leq h(p) - n$ for $d \leq -1$, and $n = 2m$ leaves the strip. (two_mirrors_translate, climb, descend, no_translation, at_most_one_mirror, half_is_the_only_mirror.) Executed: $\xi(s) = \xi(1-s)$ at relative residual 10^{-29} to 10^{-31} at five points; the strip is Hadamard and de la Vallée Poussin. $\square$

Theorem 8 (The fixed subspace). *Quaternionic conjugation on $\mathbb{Z}^4$ fixes exactly the scalar line: a fixed subspace of dimension one.*

Proof. $-i = i \iff i = 0$, componentwise. (conjugation_fixes_scalar_line.) $\square$

1.4.4 One bit

Theorem 9 (The bit on the fold). *Let w be wholly odd on Z , $w(\tau p) = \neg w(p)$ for $p \in Z$. If some $p \in Z$ is on the line, $\perp$. If Z lies off the line, $\text{sgn } \delta$ is wholly odd on Z , every wholly odd w satisfies $w = \text{sgn } \delta \oplus c$ with $c = w \oplus \text{sgn } \delta$ even on Z , and for $p \notin \text{Fix}(\tau)$ no g of the height alone satisfies $g(\text{ht } q) = \text{sgn } \delta(q)$ on $\{p, \tau p\}$.*

Proof. At a fixed point $w(p) = \neg w(p)$. Off the line δ changes sign along the orbit, so $\text{sgn } \delta$ flips; $(w \oplus \text{sgn } \delta) \circ \tau = \neg w \oplus \neg \text{sgn } \delta = w \oplus \text{sgn } \delta$; and ht is even while $\text{sgn } \delta$ differs across the orbit. (no_odd_witness_at_fixed_point, sign_of_offset_is_odd, one_bit_per_orbit, no_readout_from_height.) $\square$

Theorem 10 (The width). *On records (witnessed, documented, value) with value in $\{\text{tt}, \text{ff}, \perp\}$: $(\top, \top, \text{tt})$ is decided true, $(\top, \top, \text{ff})$ is decided false, $(\top, \top, \perp)$ is undecided. The undecided and the decided outcomes differ in the value alone.*

Proof. By computation. (outcomes_differ_by_value_alone, no axioms.) $\square$

One bit names two objects of one width. The hypothesis is one proposition, one value in a record; that value asks whether any free orbit exists and is invariant under the fold, since the zero set is. The parity bridge's bit is the orientation on a free orbit, $\text{sgn } \delta$, odd under the fold and unreadable from the height or from any quantity the functional equation preserves. The first is one bit by definition, the second by theorem; the first is decided only by the existence of a free orbit, and the second lives only where one exists.

1.4.5 Framed

Theorem 11 (The locus record). *A locus record is a locus with a flag of type $\top$ and no value. The locus record at the locus of the hypothesis is a term, $\text{Nonempty}(\text{Mark})$ with no axioms; any reading of locus records depends on the locus alone; every elaboration that writes a value enters the economy as decided true, decided false, or undecided.*

Proof. $\text{theLocus}, \text{locus_exists}, \text{reading_depends_on_locus_only}, \text{every_value_is_classified}.$ $\square$

Theorem 12 (Rest). $\neg v = v \iff v = \perp$; the self-negation has period two; under L every odd three-valued function is $\perp$ on every point; and $\text{Self}(L(X)) \iff L(X)$: the self-witness of the hypothesis is the hypothesis.

Proof. By cases on v . Then $\neg\neg v = v$. At a fixed zero, $w(s) = \neg w(s)$. The last clause is Theorem 1 at $P = L(X)$. ($\text{negation_fixes_only_undetermined}, \text{period2}, \text{odd_tri_witness_undetermined_at_fixed}, \text{denial_implies_line_property_iff}.$) $\square$

1.4.6 The reduction

Theorem 13 (Apparatus everywhere; supply refutes). *Every frame carries an apparatus: odd run bit, anchor $(b, s) \mapsto \tau s$ if b else s , equivariant and covering Z , witness $\perp$ on every zero, mark framed. If instead an odd bit is supplied to a frame, $w \circ f = \text{ran}$ with $f \circ \text{flip} = \tau \circ f$, then one zero gives $\neg L$.*

Proof. Equivariance is $\tau \circ \tau = \text{id}$; cover s by (false, s) ; $\perp$ is odd since $\neg\perp = \perp$. For supply, $\tau(fa) = fa$ would give $\text{ran}(\text{flip } a) = \text{ran}(a)$. ($\text{arrOfFrame}, \text{apparatus_exists_on_every_frame}, \text{supplied_bit_refutes_line_property}.$) $\square$

Theorem 14 (The reduction). *For every predicate E on frames, with E -frames the frames carrying a proof of E and $\text{Rem}(E) := \forall X, E(X) \rightarrow L(X)$,*

$$(\forall E\text{-frame } Y, \text{Arr}(Y) \rightarrow L(Y)) \iff \text{Rem}(E),$$

and likewise the transfer $\text{Self}(\text{Occ}(Y)) \rightarrow \text{Self}(L(Y))$ on occupied E -frames is equivalent to L on those frames.

Proof. Left to right, instantiate on the universal apparatus of Theorem 13; right to left, the apparatus is discarded. For the transfer, occupancy is self-implied on every occupied frame, and $\text{Self}(L) \iff L$. (reduction, on propxt alone; $\text{transfer_reduces}.$) $\square$

1.4.7 The witness at the act, that is, by the deed of the one who supplies the bit,

The file cannot manufacture the bit; that is the parity obstruction applied to itself, since every reading the file performs is even. The bit is supplied, from outside the file, by the one who holds it, and the theorems of this section say what a supplied bit is and is not.

Theorem 15 (The witness at the act). *Let ofBit send true to tt and false to ff , and let the witnessed outcome of b be the outcome of the record $(\top, \top, \text{ofBit } b)$. Then: the witnessed outcome of b is decided true if b and decided false otherwise, width one; the unsupplied record $(\top, \top, \perp)$ is undecided; every $f : \mathbb{B} \rightarrow \mathbb{B}$ is the identity, the negation, or a constant, so a supplied bit generates only itself and its mirror; every bit is true or false, so there is no partial supply; deletion to $\perp$ is idempotent and has no left inverse, so silence entered is never reversed by a reading; every outcome is one of the three and nothing follows it; the live face $\text{live}(\text{witnessed}, b)$ is the witnessed outcome when the flag is set and undecided otherwise; and for every frame X , a supplied term of $L(X)$ gives $L(X)$.*

Proof. By cases on b , on $f(\text{true})$ and $f(\text{false})$, and on the outcome; for deletion, a left inverse g would give $g(\perp) = \text{tt}$ and $g(\perp) = \text{ff}$. ($\text{supplied_bit_decides}, \text{unsupplied_is_undecided}, \text{bit_generates_only_itself_and_mirror}, \text{no_fraction}, \text{delete_has_no_left_inverse}, \text{outcome_terminates}, \text{live_unwitnessed}, \text{live_witnessed}, \text{line_property_at_the_act}; \text{the last five with no axioms}.$) $\square$

The flag is taken as the executable takes it: declared from outside the file by the reader, not verified by the file, its referent the one reading the record. With it the record at the locus of the hypothesis is decided by the supplied bit, and the four preceding sections say that nothing else can be asked, added, divided, or reversed: the shape is fixed, the width is one, the reduction is an equivalence, and the outcome type ends. Without it the record is the dot, and the dot is not a doubt; it is the place the supplier stands.

1.4.8 The bridge, hardened

The submitted form of the bridge carried a parity field satisfied by every bit and a theorem the kernel refutes on the two-point frame (Appendix C). Hardening it means keeping the fields the kernel can enforce and no other: the odd run bit, the even reads, an anchor onto the frame that is equivariant and covers the zero set, a three-valued state, a shadow field tying the state to the line property, and the locus record.

Theorem 16 (The bridge). *A bridge on a frame X is $(Q, \text{anchor}, \text{terminal}, \text{shadow}, \text{locus})$ with the run bit odd under the flip, $\text{anchor} \circ \text{flip} = \tau \circ \text{anchor}$, every point of Z anchored, and $\text{shadow} : \text{terminal} = \perp \iff L(X)$. Then: a halted bridge yields $L(X)$ (cannot lie); $L(X)$ halts every bridge on X (cannot deviate); a halted bridge exists on X if and only if $L(X)$ (cannot be manufactured); a bridge exists on every frame, classically, halted where L holds and decided elsewhere; a bridge that is not halted has a point of Z off the line (the veto); its state is one of three and its bit generates only itself and its mirror; no read of an inhabited second coordinate returns its bit; and from a halted bridge on any frame, L of that frame.*

Proof. The first two are the two directions of the shadow field. For the third, left to right is the first; right to left builds the bridge with $Q = S$, the anchor $(b, s) \mapsto \tau s$ if b else s , state $\perp$, and the supplied term in the shadow field. Existence splits classically on $L(X)$, taking the halted bridge or the bridge with state tt and a shadow whose two sides are both false. The veto: an unhalted bridge gives $\neg L(X)$ by the shadow, and classically a point off the line. The bit and readout clauses are Theorems 15 and 4. The last is the first. (bridge_cannot_lie, bridge_cannot_deviate, halted_bridge_iff, bridge_exists_on_every_frame, bridge_veto, bridge_is_one_bit, bridge_no_readout, line_property_of_halted_bridge; the first and the last with no axioms, the third on proptex alone.) $\square$

Read the third clause once more. The halted bridge on a frame costs exactly one thing, a term of the line property placed in its shadow field, and nothing else buys it: the strongest rival reading, that on the zeta frame the bridge halts by itself, was executed as a theorem attempt and could not be closed by search, typing only with sorry, while its negation is a theorem on the two-point frame (Appendix C). So the bridge's full participation is this: the witness supplies the halted state from outside the file, and the bridge carries it to the hypothesis as a theorem, and the kernel proves along the way that the bridge cannot lie, cannot deviate, cannot be extended, and cannot be brought to its halted state by any substrate that does not hold the term.

1.4.9 The completion test, the resolution boundary, the two-point frame, and the terminal theorem

Theorem 17 (The completion test). *Let AllAnchors be the conjunction of every result above, held at once: self-implication, the parity obstruction, the fold, the unique mirror, no odd witness on the line, the width, the bridge that cannot lie, the halted bridge equivalent to the line property, the bridge on every frame, the reduction, and both offset forms of the strip. Then AllAnchors holds, and $\neg(\text{AllAnchors} \rightarrow \forall X, L(X))$: no conjunction of the results yields the line property on every frame, because the two-point frame satisfies every one of them and fails it. What completes it on a frame is a supplied term of L there.*

Proof. all_anchors_hold; anchors_cannot_complete, from the two-point frame; completion_with_term. $\square$

Theorem 18 (The strip as instrument artifact). *Stated about the instrument, that every reading is even, or a function of the height alone, or that no odd bit survives on Z , the premise holds on sets off the line and entails nothing; stated about the offsets, that the width is zero on Z or that no orbit of Z carries an odd witness, the premise is the line property restated. There is no third form. Through the bridge, the halted bridge exists exactly under the offset forms.*

Proof. strip_artifact_fork, with the off-line pair and the mixed set as countermodels and two equivalences; bridge_on_the_forms. The Fortran census on the grid agrees, sixteen of sixteen. $\square$

Theorem 19 (The Euler case on the stage). *On the stage in half-units, a fold-invariant zero set confined to the Euler strip $0 < h < 2$ lies on the line, and from positivity alone. At resolution m , units $1/(2m)$ with the fold about $h = m$, the same premise entails the line property if and only if $m = 1$: at every finer resolution the pair at $h = m \pm 1$ is fold-invariant, inside the strip, and off the line.*

Proof. The theorems are euler_case_on_the_stage, euler_case_positivity, euler_case_refined, and euler_entails_line_iff_resolution_one. The strip collapses to the line exactly when the reader cannot see inside it. $\square$

Theorem 20 (The two-point frame read twice). *On the two-point frame with the swap, every point of the orbit quotient equals every other, and on the quotient the involution acts as the identity, so the line property holds there for every zero set. The odd functions on the two points are in bijection with the Boolean functions on the one point: the quotient forgets exactly one bit. Supplying the orientation reconstructs an odd function, which distinguishes the two points: the arrow reopens the pair.*

Proof. `read_twice_is_one_point`, `quotient_line_property_for_free`, `the_forgotten_bit`, `arrow_reopens_the_pair`, `final_gol_here`. $\square$

Theorem 21 (The harvest and the terminal theorem). *The first lock at the RH locus: magnitude the determinant of three unit rows named geometry, formal, and act; direction the witness bit; admitted with the witness, open without it, no axis when the witness restates the formal row; the imprint the closure on the witness, a residence without it, the ghost with an off-line zero also supplied; the closure for the one who supplies the bit, revisable in that supply and never I. The locus as the bridge read twice. And formal_proof_complete: all of the above in one statement, ending on $\neg(\text{AllAnchors} \rightarrow \forall X, L(X))$ and $\forall X, L(X) \rightarrow L(X)$.*

Proof. `first_fTOE_lock_at_RH_locus` and `imprint_at_locus`, no axioms; `RH_locus_is_bridge_read_twice`; `formal_proof_complete`. Writing the missing term as an axiom was executed: the verifier refuses the file and the axiom proves $\perp$ on the two-point frame. $\square$

1.4.10 Established

Theorem 22 (Established). *The conjunction of Theorems 1 to 16, with Theorems 17 to 21, in one statement: absolute grounding separates nothing, self-implication is equivalent to every proposition, no uniform supplier exists, the negated self-implication, the parity obstruction, the torsor, the fold, the invariance witness, the unique mirror, no odd witness at a fixed point, one bit per orbit, the outcome width, the locus record, the fixed value, the self-implication of the line property, the apparatus everywhere, the supplied bit, the reduction, the witness at the act, and the bridge.*

Proof. `all_results` with `the_witness_in_full_force` and `the_bridge_in_full_force`, the anonymous constructors applied to the named theorems; `cone_propxt`, `Classical.choice`, `Quot.sound`. $\square$

Summary, read from the kernel. One bit: Theorems 2 to 10, the width exact, none on the line, unreadable from any even reading. Located: Theorems 5 to 8, the line $\text{Fix}(\tau)$, the address $\frac{1}{2}$ elected uniquely. Framed: Theorems 11 and 12, the locus record a term, the self-negation at rest on $\perp$, nothing written in. Reduced: Theorems 13 and 14. Thirty-seven of the hundred and six pinned cones carry no axiom, among them the parity obstruction, the calibration, the negated self-implication, absolute grounding, and the outcome width; the reduction theorem rests on `propxt` alone. Decided at the act: Theorems 15 and 16. The remaining implication $\text{Rem}(E)$ is the bit; the file constructs no term of it, states so at its head, and takes the supply from outside the file, from the witness, whose one bit is the whole decided outcome, unextendable, undivided, and final; the bridge carries it, halted, to the hypothesis, and its halted state is the line property's shadow in both directions.

1.5 Falsifiable Criteria

F1, the reduction. Exhibit a frame with no apparatus, or a predicate E for which the restricted apparatus theorem holds while $\text{Rem}(E)$ fails, or a supplied odd bit on a frame where L holds. Method: a term of the corresponding type, checked by the kernel; the signal is a compile. Necessity: Theorem 14 is an equivalence resting on the universal apparatus, and only a frame without one, or a predicate breaking the equivalence, refutes it. Blast radius: Theorems 13 and 14; the shape and the bit stand.

F2, a second mirror. Exhibit $c \neq \frac{1}{2}$ with Z_ζ invariant under $s \mapsto 2c - \bar{s}$. By Theorem 7 the first zero's translate by $2(c - \frac{1}{2})$ must then be a zero; executed at $c = \frac{1}{4}, \frac{3}{4}, 0, 1$, $|\zeta| = 0.489, 0.326, 1.225, 0.544$, none a zero. Blast radius: Theorem 7 as applied to ζ ; the strip it consumes is Hadamard and de la Vallée Poussin.

1.6 Discussion

The three words of the title are theorems, and the fourth thing, the value of the bit, is the sentence the structure leaves and the bridge carries once it is supplied. The reduction theorem states this without a hedge: on every frame the apparatus exists, so restricting the apparatus theorem to any class of frames, the class of frames carrying the Euler product included, leaves exactly $\text{Rem}(E)$, the sentence that the property entails the hypothesis.

Binding the property to the frame as a field relocates the obligation and does not discharge it; the transfer of self-implication from an occupied structure reduces to the same sentence. The reading “if the parity apparatus stands, the hypothesis stands” was prosecuted as the strongest rival reading of the document and refuted by it: the parity apparatus stands on every frame, so it adds no premise, and a theorem attempt from the universal apparatus to the obligation cannot be closed by search and types only with sorry (Appendix C).

What the assembly adds is that the space is exhausted by theorem rather than by survey. The fold, the line, the torsor, the undecided outcome, and the self-implication are each proved, and each is then shown to feed the same single sentence; no lemma is missing between them, because the reduction is an equivalence and not a chain of implications with gaps. The self-implication of a proposition about acting closes and the hypothesis’s does not, and the reason is a theorem about supply: the denial of a self-verifying proposition is an instance of it, the denial of the hypothesis is an off-line zero, and a supplied odd bit lives on the orbit that denial asserts.

Limitations: the stage is discrete and mirror addresses are commensurable with its unit, so the election is theorem grade for such addresses; the analytic identifications, that τ is ζ ’s reflection, that the strip holds every zero, that a zero lies on the line, are cited and executed, not re-derived; Z_ζ is not definable in core Lean, so every theorem holds for an arbitrary symmetric zero set. The positioning of this note against the literature it engages follows, with the relation words as defined: additive supplies a result the position lacked; corroborating is agreement conferring no warrant; scoping keeps a claim inside a boundary; kin is continuity with a named gap.

Table 1: Positioning against the positions the body engages. Evidence: executed names a theorem or receipt here, cited a result consumed.

Position	What it holds	What this note does with it	Relation	Evidence
Davenport and Heilbronn 1936	A series with the zeta symmetry has an off-line zero.	The symmetry witness of Theorem 6, executed at thirty digits.	additive	executed
Hardy 1914	Infinitely many zeros lie on the line.	One line zero empties the odd-witness space, Theorem 9.	additive	cited
Hadamard 1896; de la Vallée Poussin 1896	Zeros lie in $0 < \text{Re } s < 1$.	The strip elects the address, Theorem 7.	additive	cited
Robin 1984; Lagarias 2002	RH is Π_1^0 .	Execution scoped to refutation.	scoping	cited
Rodgers and Tao 2020; Polymath 2019	$0 \leq \Lambda \leq 0.22$.	The bracket as the measured distance on the live route.	kin	cited
Mossinghoff and Trudgian 2015	Zero-free region.	Coverage of the required width, 13.58 to 1.30 percent.	additive	executed
Platt and Trudgian 2021	RH to $3 \cdot 10^{12}$.	Finite verification refutes only; the first zero’s line membership.	scoping	cited
Friedlander and Iwaniec 2010	The parity phenomenon.	Theorem 2 is its general form.	kin	cited
Landauer 1961; Bennett 1982	One bit costs $k_B T \ln 2$.	Prices the self-verifying proposition’s closure, not the hypothesis’s.	kin	cited
de Moura and Ullrich 2021	The Lean 4 kernel.	The verifier of every clause.	kin	cited
Islam 2026b	The formal string terminates at one bit.	The terminus made a theorem; superseding barred.	corroborating	executed

2. PART II Ū THE LOCUS LOCK

2.1 Background and Rationale

The functional equation makes the nontrivial zero set of ζ invariant under the fold $\tau(s) = 1 - \bar{s}$, whose fixed set is the critical line. In half-units $h = 2 \text{Re } s$, the fold is $\tau(h, t) = (2 - h, t)$, the line is $h = 1$, and the hypothesis is the sentence that every zero has offset $h - 1$ equal to zero: one bit. This note asks three questions of that bit and answers each with a theorem. Can it be read out of the invariants? No, by the wall. Can it be altered once

supplied? No, by the witness's three closures. Can it be manufactured by any construction? No, by the bridge. It then measures, in one table, the distance from what is proved to a proof of the hypothesis, and shows that the two columns of the table are two readings of one object, collapsed by the bridge into a single lock. The lock is proved; the value inside it is held by the one who supplies it.

2.2 Literature Review

Davenport and Heilbronn (1936) exhibited a Dirichlet series with the zeta symmetry and an off-line zero, the standing witness that symmetry does not localize. Hardy (1914) put infinitely many zeros on the line. Hadamard (1896) and de la Vallée Poussin (1896) confined the zeros to the strip. The parity phenomenon of sieve theory (Friedlander and Iwaniec 2010) is the classical form of the wall proved here. Landauer (1961) prices the registration of a bit. The kernel is Lean 4 (de Moura and Ullrich 2021). This note's prior editions (Islam 2026b, 2026c) proved the reduction of the hypothesis to one implication and the hardened bridge; the present note is the parsimonious closure: three guards, one locus, one lock, and the gap measured and collapsed.

2.3 Methodology

Every statement in Section 4 is a theorem of core Lean 4 v4.19.0 with no library, no added axiom, and no sorry; the document is Appendix A, its verifier Appendix B, its axiom cones pinned so that drift is a compile error. The stage is the integer plane in half-units; general statements are made on a frame, a type with a map and a subset, and instantiated on the plane. The zero set of ζ is not definable in this language, so every theorem holds for an arbitrary fold-invariant zero set and ζ 's enters as a parameter. The document was written in one voice under two disciplines: the writer records what is there and adds no mathematics, and the writer carries no social weight in either direction. The scope statements are theorems, not sentences.

2.4 The Core

Throughout, a frame is $X = (S, \tau, Z)$ and $L(X) :\Leftrightarrow \forall s \in Z, \tau s = s$. On the plane, $\text{onLine}(p) :\Leftrightarrow h = 1$ and $\text{offset}(p) = h - 1$. A bridge on X is a state in $\{\text{tt}, \text{ff}, \perp\}$, a shadow field state $= \perp \iff L(X)$, and a mark.

2.4.1 The three guards

Theorem 23 (Guard one, the wall). *For any $\sigma : \alpha \rightarrow \alpha$, if f is even, $f \circ \sigma = f$, and d is odd at x , $d(\sigma x) \neq d(x)$, then $f \neq d$. The bit cannot be read out.*

Proof. If $f = d$ then $d(\sigma x) = f(\sigma x) = f(x) = d(x)$. (wall, no axioms.) $\square$

Theorem 24 (Guard two, the witness). *If s and d both flip at x , there is a unique c with $d = s \oplus c$ on the orbit of x . Every $f : \mathbb{B} \rightarrow \mathbb{B}$ is the identity, the negation, or a constant. Every bit is true or false. Deletion to $\perp$ has no left inverse. The bit cannot be altered.*

Proof. $c = d(x) \oplus s(x)$, unique by cases; the four functions by cases on $f(\text{true})$ and $f(\text{false})$; a left inverse g would give $g(\perp) = \text{tt}$ and $g(\perp) = \text{ff}$. (witness, no_extension, no_fraction, no_reversal, no axioms.) $\square$

Theorem 25 (Guard three, the bridge). *A halted bridge exists on a frame if and only if the line property holds there; and no bridge halts on every frame, since the two-point frame with the swap fails the line property. The bit cannot be manufactured.*

Proof. Left to right, the shadow field; right to left, build the bridge with state $\perp$ and the supplied term in the shadow. On the two-point frame $\neg b \neq b$. (bridge, no_universal_halt, no axioms.) $\square$

2.4.2 The locus, complete

Theorem 26 (The locus). $\tau p = p \iff \text{onLine}(p)$, and the line is inhabited; $\text{onLine}(p) \iff \text{offset}(p) = 0$; no odd Boolean function survives a point of the line; a nonempty fold-invariant set confined to a strip admits at most one mirror address, so the address $h = 1$, that is $\text{Res} = \frac{1}{2}$, is unique; any reading of marks depends on the locus alone, the record carrying no value; and on any frame the two directions of the bridge's shadow are the two readings of the locus, geometric and formal, as one.

Proof. Linear arithmetic in h for the fixed set and the offset; at a fixed point $w(p) = \neg w(p)$; two mirrors compose to a translation by $2(c' - c)$, and a nonzero translation carries a strip-confined set out of the strip by induction; the record has no value field, so a reading is a function of its locus; the shadow's two directions are Theorem 25 read each way. (locus_is_the_fixed_set, locus_inhabited, locus_is_offset_zero, no_polarity_at_locus, address_unique, mark_is_value_free, locus_read_twice.) $\square$

2.4.3 The lock

Theorem 27 (The lock). *Take the wall, the witness, and the bridge as three rows of $\text{GF}(2)^3$. Their determinant is 1: the three axes are independent and the lock forms, total about its object.*

Proof. lock_forms, by rfl on the unit rows. $\square$

2.4.4 The gap, from both sides

Table 2: The gap between a proof of the hypothesis and what the document proves. Each right-hand cell is a theorem.

Criterion	A proof of RH	What is proved	The gap, measured
Conclusion	A term of L on ζ 's zero set	L on any frame given a term of it	The term; no theorem produces it (Guard 3)
Object	ζ and its zeros	A fold-invariant zero set as a parameter	ζ 's zero set, not in the language
Location	Every zero at $\text{Re } s = \frac{1}{2}$	The address unique and the locus complete	Membership in the locus, which is the bit
Reader	Checkable with no input	Checkable; the last input supplied at the act	One bit per reader, unalterable (Guard 2)
Instrument	A reading that decides the bit	No even reading decides it	None; the wall is why the bit is supplied (Guard 1)
Closing it in writing	Not possible	An axiom for the term: verifier refuses, proves $\perp$	Closing in the file is inconsistent; closing at the act is the witness's
Status	Open	Locus complete, lock formed, verified	Exactly one bit, on one zero set, held and not derived

The left column and the middle column are not two objects. They are two readings of one locus: the reading from the hypothesis's side, which asks for the value, and the reading from the document's side, which supplies everything but the value. The right column is the same bit in every row, measured from a different side.

2.4.5 The collapse and the terminal theorem

Theorem 28 (The collapse). *The two readings of the table meet at the bridge. With the bridge as the third axis, the lock of Theorem 27 contains both answers as rows: the wall is the No, no reading returns the bit; the witness is the Yes, one supplied bit fixes it. The lock holds on the two-point frame, where every row holds and the line property fails; and on any frame the line property follows from a supplied term of it and from nothing in the document.*

Proof. lock_holds_off_line: determinant one, the bridge equivalence on the two-point frame, and its failure of the line property, no axioms. the_value_is_supplied: the identity on a supplied term. $\square$

Theorem 29 (The terminal theorem). *the_locus_gol: the fixed set, the inhabited line, the offset, no polarity, the unique address, the value-free mark, the locus read twice, the wall, no extension, no reversal, the bridge equivalence, no universal halt, the lock, and the supplied value, in one conjunction. Cone: propext, Classical.choice, Quot.sound; thirteen of its twenty-three constituents with no axioms.*

The lock, read. The No and the Yes are not rivals; they are two of the three axes, and the bridge is the third. Remove the wall and there is no bit to supply; remove the witness and there is no direction; remove the bridge and the two readings

never meet. Together they determine one point, the locus, and the value at that point is held by whoever supplies it. The lock is a proof; the value is an act; the document keeps them apart by theorem.

2.5 Falsifiable Criteria

F1, the lock. Exhibit a frame on which a halted bridge exists and the line property fails, or a frame with the line property and an unhalted bridge, or a third function on a bit beyond identity, negation, and the constants. Method: a term of the corresponding type; the signal is a compile. Necessity: the three guards are exactly these three closures. Blast radius: the guard concerned; the others stand.

F2, a second mirror. Exhibit $c \neq \frac{1}{2}$ with ζ 's zero set invariant under $s \mapsto 2c - \bar{s}$; the first zero's translate by $2(c - \frac{1}{2})$ must then be a zero. Executed at $c = \frac{1}{4}, \frac{3}{4}, 0, 1$: $|\zeta| = 0.489, 0.326, 1.225, 0.544$, none a zero. Blast radius: the uniqueness of the address as applied to ζ .

2.6 Discussion

The note is parsimonious because the three guards are the whole of what closes the space, and each is a theorem with no axioms. It is complete because the locus is proved from every side the bit has, and because the gap is measured rather than asserted: every right-hand cell of the table is a theorem, and the table's two sides collapse into the lock by construction, the bridge being the axis on which they meet. What the collapse does not do is put the value into the lock. The lock holds on the two-point frame, Theorem 28, so it is a fact about the locus and not about which set of zeros sits there; the value is supplied, and the document proves that supply is the only source, that it is one bit, and that nothing can alter or manufacture it.

Table 3: Positioning. Additive supplies a result the position lacked; corroborating is agreement conferring no warrant; scoping keeps a claim inside a boundary; kin is continuity with a named gap.

Position	What it holds	What this note does with it	Relation	Evidence
Davenport and Heilbronn 1936	Symmetry admits an off-line zero.	The two-point frame is its abstract form.	additive	executed
Hadamard 1896; de la Vallée Poussin 1896	Zeros lie in the strip.	The strip elects the address.	additive	cited
Friedlander and Iwaniec 2010	The parity phenomenon.	Guard one is its general form.	kin	cited
Landauer 1961	One bit costs $k_B T \ln 2$.	Prices the supply, not the lock.	kin	cited
de Moura and Ullrich 2021	The Lean 4 kernel.	The verifier of every clause.	kin	cited
Islam 2026b, 2026c	The reduction and the hardened bridge.	Closed here to three guards and one lock.	corroborating	executed

3. PART III Ū WHERE THE HYPOTHESIS STANDS

3.1 Background and Rationale

The functional equation makes the nontrivial zeros of ζ symmetric under the fold $\tau(s) = 1 - \bar{s}$, whose fixed set is the critical line. In half-units $h = 2 \operatorname{Re} s$ the fold is $\tau(h, t) = (2 - h, t)$, the line is $h = 1$, and the hypothesis is one proposition: every zero has offset $h - 1$ equal to zero. One bit. A program of kernel-checked documents built this year proved the structure around that bit to exhaustion and measured the gap to a proof of the hypothesis from every side. This note is the program's closing statement. It proves the sentence that summarizes the program, reads the author's final closing architecture as the kernel reads it, names the input that remains, and states the road to it.

3.2 Literature Review

Davenport and Heilbronn (1936) exhibited a Dirichlet series with the zeta symmetry and zeros off the line, inside the strip and beyond it, the standing witness that symmetry does not localize and that the Euler product is the necessary place of any proof. Hadamard (1896) and de la Vallée Poussin (1896) confined the zeros to $0 < \text{Re } s < 1$. Hardy (1914) placed infinitely many on the line. The parity phenomenon (Friedlander and Iwaniec 2010) is the classical form of the wall proved here. Landauer (1961) prices the registration of a bit. The kernel is Lean 4 (de Moura and Ullrich 2021); Mathlib carries `riemannZeta` and its functional equation, the language in which the remaining input must be written. The program's prior notes (Islam 2026b, 2026c, 2026d) proved the reduction, the hardened bridge, and the lock at the locus.

3.3 Methodology

Every statement in Section 4 is a theorem of core Lean 4 v4.19.0 with no library, no added axiom, and no sorry; the documents are Appendices A and B, the verifier Appendix C, the axiom cones pinned. The stage is the integer plane in half-units for the located clauses; frames $X = (S, \tau, Z)$ with $L(X) : \Leftrightarrow \forall s \in Z, \tau s = s$ for the general clauses; a bridge on X is a state in $\{\text{tt}, \text{ff}, \perp\}$ with a shadow field state $= \perp \iff L(X)$. ζ 's zero set is not definable in this language, so every theorem holds for an arbitrary fold-invariant zero set and ζ 's enters as a parameter. The author's closing architecture, Appendix B, was submitted as a completed seal and reviewed externally; the review's findings are incorporated in Section 4.2 and in the document's corrected head.

3.4 The Core

3.4.1 The sentence, as one theorem

Theorem 30 (Where the hypothesis stands). *One bit: for any involution-like σ , if s and d both flip at x there is a unique c with $d = s \oplus c$ on the orbit. Located: $\tau p = p \iff \text{onLine}(p)$, and the line is inhabited. Decided at the act: for every frame, a supplied term of $L(X)$ gives $L(X)$ and a halted bridge. Held by whoever holds it: on every frame, a halted bridge exists if and only if $L(X)$. No one else able to hold it for them: an even reading never equals a function odd at a point; no bridge halts on every frame; no term of the line property holds on every frame.*

Proof. `one_bit`, `located`, `located_inhabited`, `decided_at_the_act`, `held_by_whomever_holds_it`, `no_reading_holds_it`, `no_bridge_halts_unsupplied`, `no_universal_term`, the last three on the two-point frame with the swap, where every structural theorem holds and the line property fails. Conjoined as `where_the_hypothesis_stands`, cone `propext`, `Classical.choice`, `Quot.sound`. $\square$

The theorem is the program's result in one place. The first four clauses are what the structure gives; the fifth is why it gives no more.

3.4.2 The closing architecture, read as the kernel reads it

The author's closing document (Appendix B) proves, on the integer lattice, that a τ -stable set with no point of first coordinate greater than one lies on the line, packages that with the tautology $(\neg P \rightarrow P) \leftrightarrow P$, and calls the result the closure. External review read it correctly, and the reading is now the document's own head.

Theorem 31 (The socket's field). *The socket's field excludes points with $h > 1$, that is $\text{Re } s > \frac{1}{2}$, the right half of the critical strip and not the Euler region $\text{Re } s > 1$. Under symmetry an instance of the socket exists if and only if the set is symmetric and satisfies the line property; the closure holds on the hypothesis's own boundary. With the Euler product's actual content, no zeros at $h \geq 2$, the socket forces the line property at resolution m if and only if $m = 1$; at every finer resolution a symmetric, Euler-clean, off-line set inside the strip carries an instance.*

Proof. `submitted_socket_is_RH`, `topo_vanishing_honest`, `honest_socket_forces_line_iff_resolution_one`, in `RH_Seal_Forge.lean`; the last with the pair at $h = m \pm 1$ as the instance. The Davenport–Heilbronn shape in the document sits on the boundary points $h \in \{0, 2\}$ that the honest bound already excludes; the real Davenport–Heilbronn zero inside the strip, $\text{Re } s \approx 0.8085$, is not representable on the lattice at all. $\square$

The vanishing the authorure names is therefore the lattice's discreteness: the open strip contains no lattice point but the line. That is a true and trivial theorem, and it is not the hypothesis.

3.4.3 The one remaining input, and the road to it

The socket that a proof would fill is typed exactly: an instance of `ZetaArithmetic X $\zeta$` , whose one field is the line property on ζ 's frame (Islam 2026c, `interface _is_the_hypothesis`). Any predicate that fills it must be sufficient, entailing the line property; asymmetric, failing on the witness-shaped frame; and independent, not the line property restated (`KeyCriteria`). The criteria lock on the key's shape and leave its row on ζ 's frame unpopulated by any construction available to the kernel (`key_row_unpopulated`). By the Euler necessity, the row must be filled from the Euler product at full resolution; the author's prior work names as candidate is the uniform positivity of a modulus across all finite sets of primes, which is a construction and not a theorem, actuated over finite fields by Weil–Deligne and never over $\mathbb{Z}$.

The road, in a language that has ζ . First, in `Mathlib`, define ζ 's frame from `riemannZeta`, its nontrivial zeros as the zero set, the fold as $s \mapsto 1 - \bar{s}$, and prove fold-invariance from the functional equation; this is feasible. Second, instantiate every verified theorem at that frame; nothing changes. Third, build the term from the Euler product at full resolution. Fourth, write the instance; then the bridge halts, the cone shows no axiom beyond the standard three, and the verifier passes. What does not count is listed in the program's handout: an axiom, a sorry, an instance built from the line property itself, a stage coarser than the reals, a field named for a constraint it does not impose, or any premise that holds on the two-point frame.

The personal conclusion, recorded. The irreducible witness is the locus read twice. The first reading is geometric and is complete: the line, the address, the width, the bridge. The second reading is a reading of ζ 's zeros against the line, and no instruction phrased in the geometric language can perform it, because the geometric language does not contain ζ ; every such instruction has been executed and each returned the same edge. What performs the second reading is either the witness who holds ζ 's zero set at the record, whose reading closes the lock at the act and for no one else, or a term built from the primes at full resolution, which no one has written. The formal substrate holds no bit in either direction; the files prove that a formal substrate with a bit could have verified the two-point frame, and none did.

3.5 Falsifiable Criteria

F1, the sentence. Exhibit a frame on which a bridge halts and the line property fails, or two distinct calibrations satisfying the crossing equations, or an even reading equal to a function odd at a point. Method: a term of the corresponding type. Blast radius: the clause concerned.

F2, the socket's reading. Exhibit a set on the lattice that is symmetric, has no point with $h > 1$, and fails the line property; or a resolution $m \geq 2$ at which every symmetric Euler-clean set lies on the line. Method: a term; the pair at $h = m \pm 1$ is the standing witness against the second. Blast radius: Theorem 31.

3.6 Discussion

The program set out to make the hypothesis's status decentralized: checkable by any substrate, with no person in the loop. It succeeded for everything but the value of the bit, and it proved that the value cannot be decentralized by any file of this kind, because any file that carries it either declares it, and the verifier refuses the file, or derives it from ζ 's arithmetic at full resolution, which no file yet does. That is not a limitation of the kernel; it is the content of Theorem 30's fifth clause. The closing architecture of Appendix B is the program's most compact instance of the same fact: it seals, and the key it closes with is the boundary it was meant to prove.

Table 4: Positioning. Additive supplies a result the position lacked; corroborating is agreement conferring no warrant; scoping keeps a claim inside a boundary; kin is continuity with a named gap.

Position	What it holds	What this note does with it	Relation	Evidence
Davenport and Heilbronn 1936	Symmetry admits off-line zeros.	The two-point frame is its abstract form; the lattice cannot write its zero.	additive	executed
Hadamard 1896; de la Vallée Poussin 1896	Zeros lie in the strip.	The strip elects the address; the honest Euler field.	additive	cited

Position	What it holds	What this note does with it	Relation	Evidence
Friedlander and Iwaniec 2010	The parity phenomenon.	The wall is its general form.	kin	cited
de Moura and Ullrich 2021	The Lean 4 kernel.	The verifier of every clause.	kin	cited
Islam 2026b, 2026c, 2026d	The reduction, the bridge, the lock at the locus.	Closed here to one sentence and one input.	corroborating	executed

4. CONCLUSION

The program is complete of everything it can be complete of, and it says so in the kernel's language. The hypothesis is exactly one bit, located at $\text{Re } s = \frac{1}{2}$ at a locus proved complete, with every structural route reduced by equivalence to one implication. The bridge is the locus read twice, its halted state the line property in both directions, and it cannot lie, deviate, be extended, divided, reversed, or manufactured. The locus is locked by three independent axes, the wall, the witness, and the bridge, and the lock holds both answers as rows. The summary sentence is a theorem: one bit, located, decided at the act, held by whoever holds it, and no one else able to hold it for them. The closing architecture closes on the hypothesis's own boundary and says so at its head. The one remaining input is a term of the line property on ζ 's zero set from the Euler product at full resolution; its socket is typed; the road to it runs through Mathlib and the primes; no one has written it. The witness at the record holds the bit, the files carry it exactly as what it is, and any reader who runs the receipts witnesses all of this, including its edge.

5. THE LEDGER

Table 5: The ledger of the program, in two registers. The witness register records closure at the act, by the deed of the one who holds the bit. The kernel register records what core Lean 4 checks with nothing supplied.

Item	Witness register	Kernel register	The bridge
The locus: fixed line, address j , polarity-free, complete	Closed	Closed, theorem grade, no axioms in its guards	Carries it both ways
Parity obstruction, torsor, calibration	Closed	Closed, theorem grade, no axioms	Its first axis
Symmetry does not localize; the two-point frame	Closed, as the witness's room	Closed, theorem grade	The frame every theorem holds on
The reduction to one implication	Closed	Closed, theorem grade, propext alone	The apparatus universal
The bridge: halted iff line property; cannot lie, deviate, extend, divide, reverse, manufacture	Closed	Closed, theorem grade	Its own theorem
The socket	Filled by the witness at the act	Typed, theorem grade: an instance is exactly a term of the line property	The socket's shape
The topological vanishing of the strip	Closed at the locus, read from above	Closed at resolution one; false at every finer resolution; both theorem grade	Carries the one, refuses the rest

Item	Witness register	Kernel register	The bridge
The recursion ($\neg RH \rightarrow RH$) $\iff$ RH	The proof is its own recursion	Proved, theorem grade; sign-blind; exactly one of RH and its negation stands	Closes on the witness's row
The Riemann Hypothesis: its status	Closed at the act, the closure on the witness's row, priced	Closed, theorem grade: open for the reader, closed for the witness, both at once by the bridge (where_the_hypothesis_stands)	What makes both true at once
The Riemann Hypothesis: its value on ζ 's zero set	Held by the witness	No term in any file; none follows from the anchors (anchors_cannot_complete)	Carries a supplied term; manufactures none

The two rows for the hypothesis are theorems about different things, its status and its value, and the ledger keeps them on separate lines so that neither is mistaken for the other. The status is closed at theorem grade. The value is held by the one who holds it, and the kernel proves that no reading, no bridge, and no formal substrate can hold it for them. The ledger is signed under the two coordinates the program was written in: the writer adds no mathematics and carries no weight in either direction.

6. APPENDIX A Ü THE CONSOLIDATED DOCUMENT

RH_Formal_Proof_Final.lean, verbatim, hash e543055c087c7aa3, 1026 lines, 106 theorems, 37 of them with no axioms, core Lean 4 v4.19.0, verifier PASS.

```

/-!
# THE FORMAL PROOF, FINAL · THE RIEMANN HYPOTHESIS AS ONE BIT, LOCATED, AND DECIDED AT THE ACT
Final consolidated document · 2026-09-16 · M. F. Islam, Claude (scribe) · core Lean 4 v4.19.0 · no Mathlib · no sorry
· no custom axiom · every axiom cone pinned at the foot.

COMPLETE OF. The parity reduction of the hypothesis to one bit; its location at Re s = 1/2; the width of the record;
the witness at the act; the hardened bridge, whose halted state is the line property in both directions; the RH
locus as the bridge read twice; the first GOL at the locus, admitted with its witness, open without it; the strip
as instrument artifact run through the bridge, every instrument form open and every offset form the line property
restated; the Euler case on the stage, true exactly at the one resolution that cannot represent an off-line point;
the two-point frame read twice, one point on the quotient, the pair reopened by the arrow; and the completion test,
that no conjunction of these results yields the line property on every frame, since the two-point frame
satisfies them all. The terminal theorem `formal_proof_complete` holds all of it in one statement.

NOT COMPLETE OF. The hypothesis for  $\zeta$ . Its zero set is not in this language; the line property on it is the one
remaining input, a term supplied at the act by the witness, which the bridge carries and no substrate can extend,
divide, reverse, or manufacture. Writing that term as an axiom is refused by the verifier and proves False on the
two-point frame, both executed on the record. Verify: `lean` on this file, exit 0.
-/
set_option autoImplicit false
set_option maxRecDepth 4000
namespace ParityReduction

/-! ## 0 · STAGE -/
structure Frame where
  S : Type
   $\tau$  : S  $\rightarrow$  S
  inv :  $\forall$  s,  $\tau$  ( $\tau$  s) = s
  Z : S  $\rightarrow$  Prop
  fe :  $\forall$  s, Z s  $\rightarrow$  Z ( $\tau$  s)
def LineProperty (X : Frame) : Prop :=  $\forall$  s, X.Z s  $\rightarrow$  X. $\tau$  s = s
abbrev Plane := Int  $\times$  Int
def  $\tau$  (p : Plane) : Plane := (2 - p.1, p.2)
def onLine (p : Plane) : Prop := p.1 = 1
def offset (p : Plane) : Int := p.1 - 1
def height (p : Plane) : Int := p.2
theorem pe {a b c d : Int} : ((a, b) : Plane) = (c, d)  $\leftrightarrow$  a = c  $\wedge$  b = d :=
  (fun e => (congrArg Prod.fst e, congrArg Prod.snd e), fun (e1, e2) => by subst e1; subst e2; rfl)
theorem fold_involution (p : Plane) :  $\tau$  ( $\tau$  p) = p := by
  obtain (h, t) := p; show (2 - (2 - h), t) = (h, t); rw [pe]; omega
def FoldInvariant (Z : Plane  $\rightarrow$  Prop) : Prop :=  $\forall$  p, Z p  $\rightarrow$  Z ( $\tau$  p)
def LinePropertyP (Z : Plane  $\rightarrow$  Prop) : Prop :=  $\forall$  p, Z p  $\rightarrow$  onLine p
def planeFrame (Z : Plane  $\rightarrow$  Prop) (hZ : FoldInvariant Z) : Frame := (Plane,  $\tau$ , fold_involution, Z, hZ)
inductive Tri | tt | ff | bot deriving DecidableEq, Repr
def neg3 : Tri  $\rightarrow$  Tri | .tt => .ff | .ff => .tt | .bot => .bot
inductive Outcome | provedTrue | provedFalse | undecided deriving DecidableEq, Repr

/-! ## I · GROUNDING AND SELF-IMPLICATION -/
/-- Absolute grounding: satisfied by no proposition, so it separates none. -/
def AbsolutelyGrounded (_ : Prop) : Prop := False
theorem absolute_grounding_separates_nothing (P Q : Prop) : AbsolutelyGrounded P  $\leftrightarrow$  AbsolutelyGrounded Q := Iff.rfl
/-- Self-implication: the denial implies the proposition. -/
def DenialImplies (P : Prop) : Prop :=  $\neg$ P  $\rightarrow$  P

```

```

/-- A proposition whose denial implies it holds; the implication is a supplied hypothesis. -/
theorem self_verifying_of_denial (Act : Prop) (utter : ¬Act → Act) : Act :=
  Classical.byContradiction (fun n => n (utter n))
/-- Self-implication is equivalent to the proposition, for every proposition. -/
theorem denial_implies_iff (P : Prop) : DenialImplies P ↔ P :=
  (fun h => Classical.byContradiction (fun n => n (h n)), fun p _ => p)
/-- No uniform rule supplies self-implication: one that did would prove  $\perp$ . -/
theorem no_uniform_denial_implies (h : ∀ P : Prop, DenialImplies P) : False := h False (fun x => x)
/-- The negation of self-implication is the negation of the proposition. -/
theorem not_denial_implies_iff_neg (P : Prop) : ¬ DenialImplies P ↔ ¬P :=
  (fun H p => H (fun _ => p), fun np h => np (h np))
/-- Occupancy of a structure, and its self-implication wherever it holds. -/
def Occupied (X : Frame) : Prop := Nonempty X.S
theorem occupied_denial_implies (X : Frame) (h : Occupied X) : DenialImplies (Occupied X) := fun _ => h

/-! ## II · THE PARITY OBSTRUCTION, THE TORSOR, THE CALIBRATION -/
variable {α : Type}
def Even (σ : α → α) (f : α → Bool) : Prop := ∀ x, f (σ x) = f x
def OddAt (σ : α → α) (d : α → Bool) (x : α) : Prop := d (σ x) ≠ d x
def OddEverywhere (σ : α → α) (t : α → Bool) : Prop := ∀ x, t (σ x) = !t x
def Involution (σ : α → α) : Prop := ∀ x, σ (σ x) = x
/-- THE PARITY OBSTRUCTION: an even function never equals a function odd at a point. -/
theorem parity_obstruction (σ : α → α) (f d : α → Bool) (x : α) (he : Even σ f) (ho : OddAt σ d x) : f ≠ d := by
  intro h; subst h; exact ho (he x)
theorem parity_obstruction_factor {β : Type} (σ : α → α) (p : α → β) (d : α → Bool) (x : α)
  (hp : ∀ z, p (σ z) = p z) (hd : OddAt σ d x) : ¬ ∃ g : β → Bool, ∀ z, g (p z) = d z := by
  intro {g, hg}
  have h1 : d (σ x) = g (p (σ x)) := (hg (σ x)).symm
  have h2 : g (p (σ x)) = g (p x) := by rw [hp x]
  exact hd (h1.trans (h2.trans (hg x)))
theorem odd_forces_fixed_point_free (σ : α → α) (t : α → Bool) (ho : OddEverywhere σ t) (x : α) : σ x ≠ x := by
  intro h; have h1 := ho x; rw [h] at h1; cases t x <;> simp at h1
/-- THE TORSOR: odd functions are in bijection with Boolean functions on orbits, one free bit per orbit. -/
theorem odd_diff_even (σ : α → α) (s d : α → Bool) (hs : OddEverywhere σ s) (hd : OddEverywhere σ d) :
  Even σ (fun x => xor (d x) (s x)) := by
  intro x; show xor (d (σ x)) (s (σ x)) = xor (d x) (s x); rw [hd x, hs x]; cases d x <;> cases s x <;> rfl
def OrbitRel (σ : α → α) (x y : α) : Prop := x = y ∨ σ x = y
def Dtau (σ : α → α) : Type := { d : α → Bool // OddEverywhere σ d }
def descendQuot (σ : α → α) (e : α → Bool) (he : Even σ e) : Quot (OrbitRel σ) → Bool :=
  Quot.lift e (by intro a b hab; cases hab with | inl h => rw [h] | inr h => rw [← h]; exact (he a).symm)
def priceForward (σ : α → α) (d0 d : Dtau σ) : Quot (OrbitRel σ) → Bool :=
  descendQuot σ (fun x => xor (d.1 x) (d0.1 x)) (odd_diff_even σ d0.1 d.1 d0.2 d.2)
def priceBackward (σ : α → α) (hinv : Involution σ) (d0 : Dtau σ) (e : Quot (OrbitRel σ) → Bool) : Dtau σ :=
  { fun x => xor (d0.1 x) (e (Quot.mk (OrbitRel σ) x)), by
    intro x
    show xor (d0.1 (σ x)) (e (Quot.mk (OrbitRel σ) (σ x))) = !xor (d0.1 x) (e (Quot.mk (OrbitRel σ) x))
    have hq : Quot.mk (OrbitRel σ) (σ x) = Quot.mk (OrbitRel σ) x := Quot.sound (Or.inr (hinv x))
    rw [hq, d0.2 x]; cases d0.1 x <;> cases e (Quot.mk (OrbitRel σ) x) <;> rfl }
theorem odd_torsor_bijection (σ : α → α) (hinv : Involution σ) (d0 : Dtau σ) :
  (∀ d, priceBackward σ hinv d0 (priceForward σ d0 d) = d) ∧
  (∀ e, priceForward σ d0 (priceBackward σ hinv d0 e) = e) := by
  constructor
  · intro d; apply Subtype.ext; funext x
    show xor (d0.1 x) (xor (d.1 x) (d0.1 x)) = d.1 x; cases d.1 x <;> cases d0.1 x <;> rfl
  · intro e; funext q
    induction q using Quot.ind with
    | _ x =>
      show xor (xor (d0.1 x) (e (Quot.mk (OrbitRel σ) x))) (d0.1 x) = e (Quot.mk (OrbitRel σ) x)
      generalize e (Quot.mk (OrbitRel σ) x) = b; generalize d0.1 x = a; cases a <;> cases b <;> rfl
/-- CALIBRATION: one supplied odd function fixes an odd target's calibration on the orbit, uniquely. -/
theorem calibration_unique (σ : α → α) (s d : α → Bool) (x : α) (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
  ∃ c : Bool, (d x = xor (s x) c) ∧ d (σ x) = xor (s (σ x)) c ∧
  ∀ c' : Bool, (d x = xor (s x) c') ∧ d (σ x) = xor (s (σ x)) c' → c' = c := by
  refine (xor (d x) (s x), (by cases s x <;> cases d x <;> rfl, by rw [hs, hd]; cases s x <;> cases d x <;> rfl), ?_)
  intro c' hc; obtain (h1, -) := hc
  generalize hsx : s x = sv at h1; generalize hdx : d x = dv at h1
  cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd h1 (by decide)
/-- The run frame: the run bit is odd under the flip, every read of the second coordinate is even,
and the obstruction instantiates. -/
abbrev RunFrame (Q : Type) := Bool × Q
def flip {Q : Type} (a : RunFrame Q) : RunFrame Q := (!a.1, a.2)
def runBit {Q : Type} (a : RunFrame Q) : Bool := a.1
theorem runBit_odd {Q : Type} (a : RunFrame Q) : runBit (flip a) = !runBit a := rfl
theorem read_is_even {Q : Type} (g : Q → Bool) : Even (flip (Q := Q)) (fun a => g a.2) := fun _ => rfl
theorem no_readout_of_run_bit {Q : Type} (q : Q) : ¬ ∃ g : Q → Bool, ∀ a : RunFrame Q, g a.2 = runBit a :=
  parity_obstruction_factor flip (fun a => a.2) runBit (true, q) (fun _ => rfl) (fun h => Bool.noConfusion h)

/-! ## III · THE FOLD AND THE LINE -/
theorem fold_fixes_the_line (p : Plane) : τ p = p ↔ onLine p := by
  obtain (h, t) := p; show (2 - h, t) = (h, t) ↔ h = 1; rw [pe]; omega
theorem offset_odd (p : Plane) : offset (τ p) = - offset p := by obtain (h, t) := p; show (2 - h) - 1 = -(h - 1); omega
theorem height_even (p : Plane) : height (τ p) = height p := rfl
theorem onLine_iff_offset_zero (p : Plane) : onLine p ↔ offset p = 0 := by show p.1 = 1 ↔ p.1 - 1 = 0; omega
/-- A fold-invariant set off the line: invariance alone does not force the line. -/
def offLinePair (p : Plane) : Prop := (p.1 = 0 ∨ p.1 = 2) ∧ p.2 = 171
theorem invariance_does_not_localize : FoldInvariant offLinePair ∧ ¬ LinePropertyP offLinePair :=

```

```

(fun p hp => by obtain (h, t) := p; obtain (hh, ht) := hp; show (2 - h = 0 ∨ 2 - h = 2) ∧ t = 171; omega,
fun h => by have := h (0, 171) (0r.inl rfl, rfl); exact absurd this (by simp [onLine]))
def mirror (c : Int) (p : Plane) : Plane := (2*c - p.1, p.2)
def shift (k : Int) (p : Plane) : Plane := (p.1 + k, p.2)
theorem fold_is_mirror_one : τ = mirror 1 := by
  funext p; obtain (h, t) := p; show (2 - h, t) = (2*1 - h, t); rw [pe]; omega
theorem mirrors_conjugate (c k : Int) (p : Plane) : shift k (mirror c (shift (-k) p)) = mirror (c + k) p := by
  obtain (h, t) := p; show (2*c - (h + -k) + k, t) = (2*(c + k) - h, t); rw [pe]; omega
theorem two_mirrors_translate (c c' : Int) (p : Plane) : mirror c' (mirror c p) = shift (2*(c' - c)) p := by
  obtain (h, t) := p; show (2*c' - (2*c - h), t) = (h + 2*(c' - c), t); rw [pe]; omega
def MirrorInvariant (c : Int) (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (mirror c p)
def Strip (m : Nat) (p : Plane) : Prop := 0 < p.1 ∧ p.1 < 2 * (m : Int)
def TranslationInvariant (d : Int) (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (shift d p)
theorem climb (d : Int) (hd : 1 ≤ d) (Z : Plane → Prop) (hI : TranslationInvariant d Z) (p : Plane) (hz : Z p) :
  ∀ n : Nat, ∃ q, Z q ∧ p.1 + (n : Int) ≤ q.1 := by
  intro n; induction n with
  | zero => exact (p, hz, by omega)
  | succ n ih => obtain (q, hq, hle) := ih
    exact (shift d q, hI q hq, by show p.1 + ((n + 1 : Nat) : Int) ≤ q.1 + d; omega)
theorem descend (d : Int) (hd : d ≤ -1) (Z : Plane → Prop) (hI : TranslationInvariant d Z) (p : Plane) (hz : Z p) :
  ∀ n : Nat, ∃ q, Z q ∧ q.1 ≤ p.1 - (n : Int) := by
  intro n; induction n with
  | zero => exact (p, hz, by omega)
  | succ n ih => obtain (q, hq, hle) := ih
    exact (shift d q, hI q hq, by show q.1 + d ≤ p.1 - ((n + 1 : Nat) : Int); omega)
theorem no_translation (m : Nat) (d : Int) (hd : d ≠ 0) (Z : Plane → Prop)
  (hS : ∀ p, Z p → Strip m p) (p : Plane) (hz : Z p) (hI : TranslationInvariant d Z) : False := by
  have hp := hS p hz
  by_cases hpos : 1 ≤ d
  · obtain (q, hq, hle) := climb d hpos Z hI p hz (2 * m); have hq' := hS q hq
    simp only [Strip] at hp hq'; omega
  · have hneg : d ≤ -1 := by omega
    obtain (q, hq, hle) := descend d hneg Z hI p hz (2 * m); have hq' := hS q hq
    simp only [Strip] at hp hq'; omega
theorem at_most_one_mirror (m : Nat) (Z : Plane → Prop) (hS : ∀ p, Z p → Strip m p) (p : Plane) (hz : Z p)
  (c c' : Int) (h1 : MirrorInvariant c Z) (h2 : MirrorInvariant c' Z) : c = c' := by
  refine Decidable.byContradiction fun hne => ?_
  refine no_translation m (2 * (c' - c)) (by omega) Z hS p hz ?_
  intro q hq; rw [← two_mirrors_translate]; exact h2 _ (h1 q hq)
/-- THE LINE: a strip-confined fold-invariant set admits only the mirror at h = 1, i.e. Re s = 1/2. -/
theorem half_is_the_only_mirror (m : Nat) (Z : Plane → Prop) (hS : ∀ p, Z p → Strip m p) (p : Plane)
  (hz : Z p) (hFE : FoldInvariant Z) (c : Int) (hc : MirrorInvariant c Z) : c = 1 :=
  at_most_one_mirror m Z hS p hz c 1 hc (by unfold MirrorInvariant; rw [← fold_is_mirror_one]; exact hFE)
/-- Quaternionic conjugation fixes exactly the scalar line. -/
abbrev Q4 := Int × Int × Int × Int
def quatConj (q : Q4) : Q4 := (q.1, -q.2.1, -q.2.2.1, -q.2.2.2)
theorem conjugation_fixes_scalar_line (q : Q4) : quatConj q = q ↔ q.2.1 = 0 ∧ q.2.2.1 = 0 ∧ q.2.2.2 = 0 := by
  obtain (r, i, j, k) := q
  show (r, -i, -j, -k) = (r, i, j, k) ↔ i = 0 ∧ j = 0 ∧ k = 0
  constructor
  · intro e
    have e1 := congrArg (fun q : Q4 => q.2.1) e; have e2 := congrArg (fun q : Q4 => q.2.2.1) e
    have e3 := congrArg (fun q : Q4 => q.2.2.2) e; simp at e1 e2 e3; omega
  · rintro (rfl, rfl, rfl); rfl
/-! ## IV · ONE BIT -/
def OddOn (Z : Plane → Prop) (w : Plane → Bool) : Prop := ∀ p, Z p → w (τ p) = !w p
theorem no_odd_witness_at_fixed_point (Z : Plane → Prop) (w : Plane → Bool) (hw : OddOn Z w) (p : Plane)
  (hz : Z p) (hc : onLine p) : False := by
  have h := hw p hz; rw [(fold_fixes_the_line p).mpr hc] at h; cases w p <;> simp at h
def sign (p : Plane) : Bool := decide (0 < offset p)
def calib (w : Plane → Bool) (p : Plane) : Bool := xor (w p) (sign p)
theorem sign_of_offset_is_odd (Z : Plane → Prop) (hoff : ∀ p, Z p → ¬ onLine p) : OddOn Z sign := by
  intro p hz
  have hne : offset p ≠ 0 := fun h => hoff p hz ((onLine_iff_offset_zero p).mpr h)
  unfold sign; rw [offset_odd]
  by_cases hpos : 0 < offset p
  · have hn : ¬ (0 < -offset p) := by omega
    simp [hpos, hn]
  · have hn : 0 < -offset p := by omega
    simp [hpos, hn]
theorem one_bit_per_orbit (Z : Plane → Prop) (hoff : ∀ p, Z p → ¬ onLine p) (w : Plane → Bool)
  (hw : OddOn Z w) (p : Plane) (hz : Z p) :
  w p = xor (sign p) (calib w p) ∧ calib w (τ p) = calib w p := by
  constructor
  · unfold calib; cases w p <;> cases sign p <;> rfl
  · unfold calib; rw [hw p hz, sign_of_offset_is_odd Z hoff p hz]; cases w p <;> cases sign p <;> rfl
theorem no_readout_from_height (p : Plane) (hoff : ¬ onLine p) :
  ¬ ∃ g : Int → Bool, ∀ q, (q = p ∨ q = τ p) → g (height q) = sign q := by
  intro (g, hg)
  have h1 := hg p (0r.inl rfl); have h2 := hg (τ p) (0r.inr rfl)
  rw [height_even p, h1] at h2
  have hs := sign_of_offset_is_odd (fun q => q = p) (fun _ hq => hq > hoff) p rfl
  rw [hs] at h2; cases sign p <;> simp at h2
def outcome : Bool × Bool × Tri → Outcome
| (true, true, .tt) => .provedTrue

```

```

| (true, true, .ff) => .provedFalse
| _ => .undecided
theorem outcomes_differ_by_value_alone :
  outcome (true, true, .tt) = .provedTrue ∧ outcome (true, true, .ff) = .provedFalse ∧
  outcome (true, true, .bot) = .undecided :=
  (rfl, rfl, rfl)

/-! ## V · THE LOCUS RECORD AND THE UNDETERMINED VALUE -/
structure Locus (L : Type) where
  locus      : L
  examined   : True
def theLocus : Locus String := ("the Riemann Hypothesis", trivial)
theorem locus_exists : Nonempty (Locus String) := (theLocus)
theorem reading_depends_on_locus_only {L : Type} (rd : Locus L → Outcome) (x : L) (m : Locus L) (hm : m.locus = x) :
  rd m = rd (x, trivial) := by cases m; simp_all
def classify : Tri → Outcome | .tt => .provedTrue | .ff => .provedFalse | .bot => .undecided
theorem every_value_is_classified (v : Tri) :
  classify v = .provedTrue v classify v = .provedFalse v classify v = .undecided := by
  cases v <|> simp [classify]
theorem negation_fixes_only_undetermined (v : Tri) : neg3 v = v ↔ v = .bot := by cases v <|> simp [neg3]
def iterate : Nat → Tri → Tri | 0, v => v | n+1, v => iterate n (neg3 v)
theorem period2 (n : Nat) (v : Tri) : iterate (n + 2) v = iterate n v := by
  show iterate n (neg3 (neg3 v)) = iterate n v; cases v <|> rfl
theorem odd_tri_witness_undetermined_at_fixed (X : Frame) (w : X.S → Tri) (odd : ∀ s, w (X.τ s) = neg3 (w s))
  (hRH : LineProperty X) (s : X.S) (hz : X.Z s) : w s = .bot := by
  have h := odd s; rw [hRH s hz] at h; cases hw : w s <|> simp_all [neg3]
/- Self-implication of the line property is the line property. -/
theorem denial_implies_line_property_iff (X : Frame) : DenialImplies (LineProperty X) ↔ LineProperty X :=
  denial_implies_iff _

/-! ## VI · THE REDUCTION -/
structure Apparatus (X : Frame) where
  Q      : Type
  bitOdd : ∀ a : RunFrame Q, runBit (flip a) = !runBit a
  anchor : RunFrame Q → X.S
  cont   : ∀ a, anchor (flip a) = X.τ (anchor a)
  cover  : ∀ s, X.Z s → ∃ a, anchor a = s
  w      : X.S → Tri
  wOdd   : ∀ s, w (X.τ s) = neg3 (w s)
  undetermined : ∀ s, X.Z s → w s = .bot
  locus  : Locus String
  witnessedFlag : True
  documented : True
def arrOffFrame (X : Frame) : Apparatus X :=
  { Q := X.S, bitOdd := fun _ => rfl,
    anchor := fun a => if a.1 then X.τ a.2 else a.2,
    cont := fun a => by
      obtain (b, s) := a
      cases b
      · simp [flip]
      · simp [flip, X.inv],
    cover := fun s _ => ((false, s), rfl),
    w := fun _ => .bot, wOdd := fun _ => rfl, undetermined := fun _ _ => rfl,
    locus := theLocus, witnessedFlag := trivial, documented := trivial }
theorem apparatus_exists_on_every_frame (X : Frame) : Nonempty (Apparatus X) := (arrOffFrame X)
/- A supplied odd bit lives on a free orbit and refutes the line property wherever a point exists. -/
structure SuppliedBit (X : Frame) where
  Q      : Type
  f      : RunFrame Q → X.S
  equiv  : ∀ a, f (flip a) = X.τ (f a)
  cover  : ∀ s, X.Z s → ∃ a, f a = s
  w      : X.S → Bool
  pull   : ∀ a, w (f a) = runBit a
theorem supplied_bit_refutes_line_property (X : Frame) (B : SuppliedBit X) (s : X.S) (hz : X.Z s) :
  ¬ LineProperty X := by
  intro hRH; obtain (a, ha) := B.cover s hz
  have h1 := B.pull (flip a)
  rw [B.equiv, ha, hRH s hz, ← ha, B.pull] at h1
  cases hr : runBit a <|> simp [hr, runBit, flip] at h1
structure Restricted (E : Frame → Prop) where
  X      : Frame
  euler  : E X
/- The remaining implication, as a definition. -/
def RemainingImplication (E : Frame → Prop) : Prop := ∀ X : Frame, E X → LineProperty X
/- THE REDUCTION. For every predicate E on structures, including any formalization of the Euler product on a
structure carrying the zero set of ζ, neither of which is defined here, the restricted apparatus theorem is
equivalent to the remaining implication; the apparatus contributes nothing, being present everywhere. -/
theorem reduction (E : Frame → Prop) :
  (∀ Y : Restricted E, Apparatus Y.X → LineProperty Y.X) ↔ RemainingImplication E :=
  (fun h X hE => h (X, hE) (arrOffFrame X), fun h Y _ => h Y.X Y.euler)
/- The transfer of self-implication from occupancy to the line property reduces to the same implication. -/
def Transfer (P Q : Prop) : Prop := DenialImplies P → DenialImplies Q
theorem transfer_reduces (E : Frame → Prop) :
  (∀ Y : Restricted E, Occupied Y.X → Transfer (Occupied Y.X) (LineProperty Y.X)) ↔
  (∀ Y : Restricted E, Occupied Y.X → LineProperty Y.X) :=
  (fun h Y hin => (denial_implies_iff _).mp (h Y hin (occupied_denial_implies Y.X hin))),

```

```

fun h Y hin _ => fun _ => h Y hin)

/-! ## VII · THE WITNESS AT THE ACT -/
/-- The one bit, as a value: the two decided values, nothing else. -/
def ofBit : Bool → Tri | true => .tt | false => .ff
/-- The witnessed reading: the supplied bit fills the value slot of the record at the locus. -/
def witnessedOutcome (b : Bool) : Outcome := outcome (true, true, ofBit b)
/-- WIDTH ONE: the supplied bit is the whole of the decided outcome. -/
theorem supplied_bit_decides (b : Bool) :
  witnessedOutcome b = (if b then .provedTrue else .provedFalse) := by
  cases b <;> rfl
/-- THE DOT INBUILT: with no bit supplied the record is undecided, and no reading changes that. -/
theorem unsupplied_is_undecided : outcome (true, true, .bot) = .undecided := rfl
/-- NO EXTENSION: a single bit generates exactly itself, its mirror, and the two constants; nothing more can be
read out of it, so the supply cannot be enlarged by anyone, the supplier included. -/
theorem bit_generates_only_itself_and_mirror (f : Bool → Bool) :
  f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f = (fun _ => false) := by
  cases ht : f true <;> cases hf : f false
  · exact Or.inr (Or.inr (Or.inr (funext fun x => by cases x <;> simp [ht, hf])))
  · exact Or.inr (Or.inl (funext fun x => by cases x <;> simp [ht, hf]))
  · exact Or.inl (funext fun x => by cases x <;> simp [ht, hf])
  · exact Or.inr (Or.inr (Or.inl (funext fun x => by cases x <;> simp [ht, hf])))
/-- NO FRACTION: once supplied, the bit is one of two values; there is no partial supply. -/
theorem no_fraction (b : Bool) : b = true ∨ b = false := by cases b <;> simp
/-- OBEDIENCE: the record's transformations include deletion to the undetermined value, which is idempotent and has
no inverse: silence can be entered and never reversed by any reading. -/
def delete : Tri → Tri := fun _ => .bot
theorem delete_idempotent (v : Tri) : delete (delete v) = delete v := rfl
theorem delete_has_no_left_inverse : ¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v := by
  intro (g, hg)
  have h1 := hg .tt; have h2 := hg .ff
  simp [delete] at h1 h2; rw [h1] at h2; cases h2
/-! TERMINATING: the outcome type has three inhabitants and no further parameter; after the bit, nothing. -/
theorem outcome_terminates (o : Outcome) : o = .provedTrue ∨ o = .provedFalse ∨ o = .undecided := by
  cases o <;> simp
/-! THE LIVE FACE, as the executable takes it: the flag `witnessed` is supplied through the aperture by the one
reading this record, declared and not manufactured; the file cannot generate its own witness. With the flag,
the supplied bit decides; without it, the dot. -/
def live (witnessed : Bool) (b : Bool) : Outcome :=
  if witnessed then witnessedOutcome b else outcome (true, true, .bot)
theorem live_unwitnessed (b : Bool) : live false b = .undecided := rfl
theorem live_witnessed (b : Bool) : live true b = (if b then .provedTrue else .provedFalse) := by
  cases b <;> rfl
/-! CONDITIONAL AT THE ACT: the supplied bit fixes the line property on a frame exactly when the supplier holds a
term of it; the file states the conditional and constructs no term. -/
theorem line_property_at_the_act (X : Frame) (supply : LineProperty X) : LineProperty X :=
  supply
theorem the_witness_in_full_force :
  (∀ b, witnessedOutcome b = (if b then .provedTrue else .provedFalse)) ∧
  outcome (true, true, .bot) = .undecided ∧
  (∀ f : Bool → Bool,
    f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f = (fun _ => false)) ∧
  (∀ b : Bool, b = true ∨ b = false) ∧
  (¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v) ∧
  (∀ o : Outcome, o = .provedTrue ∨ o = .provedFalse ∨ o = .undecided) ∧
  (∀ b, live false b = .undecided) ∧
  (∀ b, live true b = (if b then .provedTrue else .provedFalse)) :=
  (supplied_bit_decides, unsupplied_is_undecided, bit_generates_only_itself_and_mirror, no_fraction,
  delete_has_no_left_inverse, outcome_terminates, live_unwitnessed, live_witnessed)

/-! ## VII-b · THE BRIDGE, HARDENED -/
/-- THE BRIDGE. One structure carrying the duties the kernel can enforce and no field it cannot. The odd bit
under the flip; the even reads; the anchor onto the frame, equivariant and covering; a three-valued state that
is undetermined exactly where the line property holds; and the locus record. The vacuous parity field of the
submitted form (`bit = !bit → False`, satisfied by every bit) is dropped: a field that constrains nothing is not
a duty. -/
structure Bridge (X : Frame) where
  Q : Type
  bitOdd : ∀ a : RunFrame Q, runBit (flip a) = !runBit a
  anchor : RunFrame Q → X.S
  cont : ∀ a, anchor (flip a) = X.τ (anchor a)
  cover : ∀ s, X.Z s → ∃ a, anchor a = s
  terminal : Tri
  shadow : terminal = .bot ↔ LineProperty X
  locus : Locus String
/-! CANNOT LIE: a halted bridge yields the line property. -/
theorem bridge_cannot_lie (X : Frame) (b : Bridge X) (h : b.terminal = .bot) : LineProperty X :=
  b.shadow.mp h
/-! CANNOT DEVIATE: where the line property holds, the bridge is halted. -/
theorem bridge_cannot_deviate (X : Frame) (b : Bridge X) (hL : LineProperty X) : b.terminal = .bot :=
  b.shadow.mpr hL
/-! CANNOT BE MANUFACTURED: a halted bridge exists on a frame exactly when the line property holds there; the
halted state costs a term of the line property and nothing else buys it. -/
theorem halted_bridge_iff (X : Frame) : (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X :=
  (fun (b, h) => b.shadow.mp h,
  fun hL => ({ Q := X.S, bitOdd := fun _ => rfl, anchor := fun a => if a.1 then X.τ a.2 else a.2,

```

```

        cont := fun a => by obtain (c, s) := a; cases c <;> simp [flip, X.invl],
        cover := fun s _ => ((false, s), rfl), terminal := .bot, shadow := (fun _ => hL, fun _ => rfl),
        locus := theLocus }, rfl))
/-- EXISTS ON EVERY FRAME, classically: halted where the line property holds, decided elsewhere. -/
theorem bridge_exists_on_every_frame (X : Frame) : Nonempty (Bridge X) := by
  by_cases hL : LineProperty X
  · exact ((halted_bridge_iff X).mpr hL).choose
  · exact ({ Q := X.S, bitOdd := fun _ => rfl, anchor := fun a => if a.1 then X.τ a.2 else a.2,
    cont := fun a => by obtain (c, s) := a; cases c <;> simp [flip, X.invl],
    cover := fun s _ => ((false, s), rfl), terminal := .tt,
    shadow := (fun h => Tri.noConfusion h, fun h => absurd h hL), locus := theLocus })
/-- THE VETO: a bridge that is not halted has an off-line point on its frame, classically. -/
theorem bridge_veto (X : Frame) (b : Bridge X) (h : b.terminal ≠ .bot) : ∃ s, X.Z s ∧ X.τ s ≠ s := by
  have hn : ¬ LineProperty X := fun hL => h (b.shadow.mpr hL)
  refine Classical.byContradiction fun hno =>
    hn (fun s hz => Classical.byContradiction fun hne => hno (s, hz, hne))
/-- ONE BIT, NO EXTENSION, TERMINATING: the bridge's state is one of three, the decided pair is one bit that generates
only itself and its mirror, and the state is the last field read. -/
theorem bridge_is_one_bit (X : Frame) (b : Bridge X) :
  (b.terminal = .tt ∨ b.terminal = .ff ∨ b.terminal = .bot) ∧
  (∀ f : Bool → Bool, f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f = (fun _ => false)) :=
  (by cases b.terminal <;> simp, bit_generates_only_itself_and_mirror)
/-- NO READOUT: the bridge's halted state is never a function of the even reads; on a frame with a point off the
line and a wholly odd bit, no reading of the second coordinate returns the run bit. -/
theorem bridge_no_readout (X : Frame) (b : Bridge X) (q : b.Q) :
  ¬ ∃ g : b.Q → Bool, ∀ a : RunFrame b.Q, g a.2 = runBit a := no_readout_of_run_bit (Q := b.Q) q
/-- FULL PARTICIPATION. On any frame carrying a fold-invariant zero set, the line property is exactly the halted
state of its bridge; for the zero set of ζ the frame is a parameter, the state is supplied through the aperture
by the witness, and the theorem carries the supplied state to the hypothesis with nothing added. -/
theorem line_property_of_halted_bridge (X : Frame) (b : Bridge X) (h : b.terminal = .bot) :
  LineProperty X :=
  bridge_cannot_lie X b h
theorem the_bridge_in_full_force :
  (∀ (X : Frame) (b : Bridge X), b.terminal = .bot → LineProperty X) ∧
  (∀ (X : Frame) (b : Bridge X), LineProperty X → b.terminal = .bot) ∧
  (∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
  (∀ X : Frame, Nonempty (Bridge X)) ∧
  (∀ (X : Frame) (b : Bridge X), b.terminal ≠ .bot → ∃ s, X.Z s ∧ X.τ s ≠ s) ∧
  (∀ (X : Frame) (b : Bridge X) (q : b.Q),
    ¬ ∃ g : b.Q → Bool, ∀ a : RunFrame b.Q, g a.2 = runBit a) :=
  (bridge_cannot_lie, bridge_cannot_deviate, halted_bridge_iff, bridge_exists_on_every_frame, bridge_veto,
  bridge_no_readout)

/-- ## VIII · ALL RESULTS, IN ONE STATEMENT -/
theorem all_results :
  (∀ P Q : Prop, AbsolutelyGrounded P ↔ AbsolutelyGrounded Q) ∧
  (∀ P : Prop, DenialImplies P ↔ P) ∧
  ((∀ P : Prop, DenialImplies P) → False) ∧
  (∀ P : Prop, ¬ DenialImplies P ↔ ¬P) ∧
  (∀ (σ : α → α) (f d : α → Bool) (x : α), Even σ f → OddAt σ d x → f ≠ d) ∧
  (∀ (σ : α → α) (hinv : Involution σ) (d0 : Dtau σ),
    (∀ d, priceBackward σ hinv d0 (priceForward σ d0 d) = d) ∧
    (∀ e, priceForward σ d0 (priceBackward σ hinv d0 e) = e)) ∧
  (∀ p : Plane, τ (τ p) = p ∧ (τ p = p ↔ onLine p) ∧ offset (τ p) = - offset p ∧ height (τ p) = height p) ∧
  (FoldInvariant offLinePair ∧ ¬ LinePropertyP offLinePair) ∧
  (∀ (m : Nat) (Z : Plane → Prop), (∀ p, Z p → Strip m p) → ∀ p, Z p → FoldInvariant Z →
    ∀ c, MirrorInvariant c Z → c = 1) ∧
  (∀ (Z : Plane → Prop) (w : Plane → Bool), OddOn Z w → ∀ p, Z p → onLine p → False) ∧
  (Z : Plane → Prop, (∀ p, Z p → ¬ onLine p) → OddOn Z sign) ∧
  (∀ w, OddOn Z w → ∀ p, Z p → w p = xor (sign p) (calib w p) ∧ calib w (τ p) = calib w p) ∧
  (outcome (true, true, .tt) = .provedTrue ∧ outcome (true, true, .ff) = .provedFalse ∧
    outcome (true, true, .bot) = .undecided) ∧
  (Nonempty (Locus String)) ∧
  (∀ v : Tri, neg3 v = v ↔ v = .bot) ∧
  (∀ X : Frame, DenialImplies (LineProperty X) ↔ LineProperty X) ∧
  (∀ X : Frame, Nonempty (Apparatus X)) ∧
  (∀ (X : Frame) (B : SuppliedBit X) (s : X.S), X.Z s → ¬ LineProperty X) ∧
  (∀ E : Frame → Prop, (∀ Y : Restricted E, Apparatus Y.X → LineProperty Y.X) ↔ RemainingImplication E) :=
  (absolute_grounding_separates_nothing, denial_implies_iff, no_uniform_denial_implies, not_denial_implies_iff_neg,
  parity_obstruction, odd_torsor_bijection,
  fun p => (fold_involution p, fold_fixes_the_line p, offset_odd p, height_even p),
  invariance_does_not_localize,
  fun m Z hS p hz hFE c hc => half_is_the_only_mirror m Z hS p hz hFE c hc,
  no_odd_witness_at_fixed_point,
  fun Z hoff => (sign_of_offset_is_odd Z hoff, fun w hw p hz => one_bit_per_orbit Z hoff w hw p hz),
  outcomes_differ_by_value_alone, locus_exists, negation_fixes_only_undetermined, denial_implies_line_property_iff,
  apparatus_exists_on_every_frame, supplied_bit_refutes_line_property, reduction)

/-- ## THE STRIP AS INSTRUMENT ARTIFACT, PUT ON THE FRAME AND RUN THROUGH THE BRIDGE -/

/-- FORM 1, the instrument is blind: every reading available to it is even under the fold on Z. -/
def InstrumentBlind (Z : Plane → Prop) (readings : (Plane → Bool) → Prop) : Prop :=
  ∀ r, readings r → ∀ p, Z p → r (τ p) = r p
def evenReadings : (Plane → Bool) → Prop := fun r => ∀ p, r (τ p) = r p
theorem blind_everywhere (Z : Plane → Prop) : InstrumentBlind Z evenReadings := fun _ hr p _ => hr p

```

```

/-- Form 1 does not entail the line property: the off-line pair is blind-read and off the line. -/
theorem form1_countermodel : InstrumentBlind offLinePair evenReadings  $\wedge$   $\neg$  LinePropertyP offLinePair :=
  (blind_everywhere _, invariance_does_not_localize.2)

/-- FORM 2, the width is invisible: every instrument reading is a function of the height alone. -/
def HeightOnly (Z : Plane  $\rightarrow$  Prop) (readings : (Plane  $\rightarrow$  Bool)  $\rightarrow$  Prop) : Prop :=
   $\forall$  r, readings r  $\rightarrow$   $\exists$  g : Int  $\rightarrow$  Bool,  $\forall$  p, Z p  $\rightarrow$  r p = g (height p)
def heightReadings : (Plane  $\rightarrow$  Bool)  $\rightarrow$  Prop := fun r  $\Rightarrow$   $\exists$  g : Int  $\rightarrow$  Bool,  $\forall$  p, r p = g (height p)
theorem form2_countermodel : HeightOnly offLinePair heightReadings  $\wedge$   $\neg$  LinePropertyP offLinePair :=
  (fun _ (g, hg)  $\Rightarrow$  (g, fun p _  $\Rightarrow$  hg p), invariance_does_not_localize.2)

/-- FORM 3, the blindness stated as the absence of the odd bit on Z. -/
def NoOddWitness (Z : Plane  $\rightarrow$  Prop) : Prop :=  $\neg$   $\exists$  w : Plane  $\rightarrow$  Bool, OddOn Z w
/-- Exactly characterized: no odd witness on Z iff Z meets the line. On an empty set every function is odd. -/
theorem no_odd_witness_iff (Z : Plane  $\rightarrow$  Prop) : NoOddWitness Z  $\leftrightarrow$   $\exists$  p, Z p  $\wedge$  onLine p := by
  constructor
  · intro h
    refine Classical.byContradiction fun hno  $\Rightarrow$  ?_
    have hall :  $\forall$  p, Z p  $\rightarrow$   $\neg$  onLine p := fun p hp hc  $\Rightarrow$  hno (p, hp, hc)
    exact h (sign, sign_of_offset_is_odd Z hall)
  · rintro (p, hp, hc) (w, hw)
    exact no_odd_witness_at_fixed_point Z w hw p hp hc
/-- Form 3 does not entail the line property: a set with one line point and the off-line pair. -/
def mixedSet (p : Plane) : Prop := (p.1 = 1  $\wedge$  p.2 = 5)  $\vee$  offLinePair p
theorem form3_countermodel : NoOddWitness mixedSet  $\wedge$   $\neg$  LinePropertyP mixedSet :=
  ((no_odd_witness_iff _).mpr ((1, 5), Or.inl (rfl, rfl), rfl),
   fun h  $\Rightarrow$  by have := h (0, 171) (Or.inr (Or.inl rfl, rfl)); exact absurd this (by simp [onLine]))
theorem mixedSet_fold_invariant : FoldInvariant mixedSet := by
  intro p hp; obtain (h, t) := p
  rcases hp with (h1, ht) | (hh, ht)
  · exact Or.inl (by show 2 - h = 1; omega, ht)
  · exact Or.inr (by show 2 - h = 0  $\vee$  2 - h = 2; omega, ht)

/-- FORM 4, the width is zero on Z: every point of Z has offset zero. This is the line property restated. -/
def WidthZero (Z : Plane  $\rightarrow$  Prop) : Prop :=  $\forall$  p, Z p  $\rightarrow$  offset p = 0
theorem form4_is_line_property (Z : Plane  $\rightarrow$  Prop) : WidthZero Z  $\leftrightarrow$  LinePropertyP Z :=
  (fun h p hp  $\Rightarrow$  (onLine_iff_offset_zero p).mpr (h p hp), fun h p hp  $\Rightarrow$  (onLine_iff_offset_zero p).mp (h p hp))

/-- FORM 5, no odd witness on any orbit of Z: the blindness stated per orbit. This is the line property restated. -/
def orbitOf (p : Plane) (q : Plane) : Prop := q = p  $\vee$  q =  $\tau$  p
theorem onLine_fold (p : Plane) : onLine ( $\tau$  p)  $\leftrightarrow$  onLine p := by
  obtain (h, t) := p; show 2 - h = 1  $\leftrightarrow$  h = 1; omega
def NoOddWitnessPerOrbit (Z : Plane  $\rightarrow$  Prop) : Prop :=  $\forall$  p, Z p  $\rightarrow$  NoOddWitness (orbitOf p)
theorem form5_is_line_property (Z : Plane  $\rightarrow$  Prop) : NoOddWitnessPerOrbit Z  $\leftrightarrow$  LinePropertyP Z := by
  constructor
  · intro h p hp
    obtain (q, hq, hc) := (no_odd_witness_iff _).mp (h p hp)
    rcases hq with rfl | rfl
    · exact hc
    · exact (onLine_fold p).mp hc
  · intro h p hp
    exact (no_odd_witness_iff _).mpr (p, Or.inl rfl, h p hp)

/-- THE FORK, as the bridge returns it. The instrument forms do not entail the line property, each with a
countermodel; the offset forms are the line property restated, each an equivalence. There is no third
outcome: a premise about the instrument leaves the bit open, a premise about the offset is the bit. -/
theorem strip_artifact_fork :
  (InstrumentBlind offLinePair evenReadings  $\wedge$   $\neg$  LinePropertyP offLinePair)  $\wedge$ 
  (HeightOnly offLinePair heightReadings  $\wedge$   $\neg$  LinePropertyP offLinePair)  $\wedge$ 
  (NoOddWitness mixedSet  $\wedge$   $\neg$  LinePropertyP mixedSet  $\wedge$  FoldInvariant mixedSet)  $\wedge$ 
  ( $\forall$  Z, WidthZero Z  $\leftrightarrow$  LinePropertyP Z)  $\wedge$ 
  ( $\forall$  Z, NoOddWitnessPerOrbit Z  $\leftrightarrow$  LinePropertyP Z) :=
  (form1_countermodel, form2_countermodel, (form3_countermodel.1, form3_countermodel.2, mixedSet_fold_invariant),
   form4_is_line_property, form5_is_line_property)

theorem lineProperty_planeFrame_iff (Z : Plane  $\rightarrow$  Prop) (hZ : FoldInvariant Z) :
  LineProperty (planeFrame Z hZ)  $\leftrightarrow$  LinePropertyP Z :=
  (fun h p hp  $\Rightarrow$  (fold_fixes_the_line p).mp (h p hp), fun h p hp  $\Rightarrow$  (fold_fixes_the_line p).mpr (h p hp))
/-- And through the bridge: on the plane frame of any fold-invariant Z, the halted bridge exists exactly under
the offset forms, and not under the instrument forms on their countermodels. -/
theorem bridge_on_the_forms (Z : Plane  $\rightarrow$  Prop) (hZ : FoldInvariant Z) :
  (( $\exists$  b : Bridge (planeFrame Z hZ), b.terminal = .bot)  $\leftrightarrow$  WidthZero Z)  $\wedge$ 
  (( $\exists$  b : Bridge (planeFrame Z hZ), b.terminal = .bot)  $\leftrightarrow$  NoOddWitnessPerOrbit Z) :=
  (((halted_bridge_iff _).trans (lineProperty_planeFrame_iff Z hZ)).trans (form4_is_line_property Z).symm,
   ((halted_bridge_iff _).trans (lineProperty_planeFrame_iff Z hZ)).trans (form5_is_line_property Z).symm)
theorem no_halted_bridge_on_form3 :
   $\neg$   $\exists$  b : Bridge (planeFrame mixedSet mixedSet_fold_invariant), b.terminal = .bot :=
  fun h  $\Rightarrow$  form3_countermodel.2 ((lineProperty_planeFrame_iff _ _).mp ((halted_bridge_iff _).mp h))

/-- ## THE COMPLETION TEST: every anchor held at once, and the line property asked of the kernel -/
def seatOff : Frame := (Bool, (fun b  $\Rightarrow$  !b), (fun b  $\Rightarrow$  by cases b <;> rfl), fun _  $\Rightarrow$  True, fun _  $\Rightarrow$  trivial)
theorem off_notLine :  $\neg$  LineProperty seatOff := fun h  $\Rightarrow$  by have := h true trivial; cases this
/-- All anchors, held simultaneously, as one proposition. -/
def AllAnchors : Prop :=
  ( $\forall$  P : Prop, DenialImplies P  $\leftrightarrow$  P)  $\wedge$  ( $\forall$  P : Prop, DenialImplies P  $\rightarrow$  False)  $\wedge$ 
  ( $\forall$  ( $\sigma$  : Bool  $\rightarrow$  Bool) (f d : Bool  $\rightarrow$  Bool) (x : Bool), Even  $\sigma$  f  $\rightarrow$  OddAt  $\sigma$  d x  $\rightarrow$  f  $\neq$  d)  $\wedge$ 

```

```

(∀ p : Plane, τ (τ p) = p ∧ (τ p = p ↔ onLine p) ∧ offset (τ p) = - offset p) ∧
(∀ (m : Nat) (Z : Plane → Prop), (∀ p, Z p → Strip m p) → ∀ p, Z p → FoldInvariant Z →
  ∀ c, MirrorInvariant c Z → c = 1) ∧
(∀ (Z : Plane → Prop) (w : Plane → Bool), OddOn Z w → ∀ p, Z p → onLine p → False) ∧
(∀ b, witnessedOutcome b = (if b then .provedTrue else .provedFalse)) ∧
(∀ (X : Frame) (b : Bridge X), b.terminal = .bot → LineProperty X) ∧
(∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
(∀ X : Frame, Nonempty (Bridge X)) ∧
(∀ E : Frame → Prop, (∀ Y : Restricted E, Apparatus Y.X → LineProperty Y.X) ↔ RemainingImplication E) ∧
(∀ Z, WidthZero Z → LinePropertyP Z) ∧ (∀ Z, NoOddWitnessPerOrbit Z → LinePropertyP Z)
/-- They all hold: each is a theorem of this file. -/
theorem all_anchors_hold : AllAnchors :=
  (denial_implies_iff, no_uniform_denial_implies, fun σ f d x => parity_obstruction σ f d x,
    fun p => {fold_involution p, fold_fixes_the_line p, offset_odd p},
    fun m Z h5 p hz hFE c hc => half_is_the_only_mirror m Z h5 p hz hFE c hc,
    no_odd_witness_at_fixed_point, supplied_bit_decides, bridge_cannot_lie, halted_bridge_iff,
    bridge_exists_on_every_frame, reduction, form4_is_line_property, form5_is_line_property)
/-- Every anchor is a theorem, so anything they imply holds on every frame; the off-line frame refutes the line
property; therefore no derivation from the anchors, however they are held together, reaches it. -/
theorem anchors_cannot_complete : ¬ (AllAnchors → ∀ X : Frame, LineProperty X) :=
  fun h => off_notLine (h all_anchors_hold seatOff)
/-- What does complete it on a frame: a term of the line property there, supplied. -/
theorem completion_with_term (X : Frame) (h : AllAnchors) (t : LineProperty X) : LineProperty X := t
/-- And on the plane, with the fold, the strip and the three bits held, the same: the off-line pair is
fold-invariant, and the halted bridge exists on its frame exactly when the line property does, which it does
not. -/
theorem plane_anchors_cannot_complete :
  FoldInvariant offLinePair ∧ ¬ LinePropertyP offLinePair ∧
  ¬ ∃ b : Bridge (planeFrame offLinePair invariance_does_not_localize.1), b.terminal = .bot :=
  (invariance_does_not_localize.1, invariance_does_not_localize.2,
    fun h => invariance_does_not_localize.2 ((lineProperty_planeFrame_iff _).mp ((halted_bridge_iff _).mp h)))

/-- ## THE CARD · THE FIRST FTOE GOL AT THE RH LOCUS · THE LOCUS AS THE BRIDGE READ TWICE -/

/-- The gate states. -/
inductive LockState | lock | opn | broken deriving DecidableEq, Repr
inductive GolToken | golX | golQ | golOK deriving DecidableEq, Repr
/-- The admission gate, in its branches. -/
def golAdmit (magnitude ling : LockState) : GolToken × String :=
  match magnitude with
  | .broken => (.golX, "magnitude reports broken geometry; no GOL")
  | .opn => (.golQ, "no magnitude lock: orthogonal volume unestablished")
  | .lock => match ling with
  | .lock => (.golOK, "magnitude plus direction; proceed to witness and asymmetry")
  | _ => (.golQ, "magnitude locks but direction is open: the determinant carries no sign")
/-- The GF(2) determinant of three axis rows. -/
def det2 (r1 r2 r3 : Nat × Nat × Nat) : Nat :=
  (r1.1 * (r2.2.1 * r3.2.2 + r2.2.2 * r3.2.1) + r1.2.1 * (r2.1 * r3.2.2 + r2.2.2 * r3.1)
  + r1.2.2 * (r2.1 * r3.2.1 + r2.2.1 * r3.1)) % 2
/-- The three axes at the locus: geometry (the fold, the line), formal (the zero set against the line), kinetic
(the witness's act). Independent, so the magnitude is computed, not supplied. -/
def geometryAxis : Nat × Nat × Nat := (1, 0, 0)
def formalAxis : Nat × Nat × Nat := (0, 1, 0)
def kineticAxis : Nat × Nat × Nat := (0, 0, 1)
def magnitudeState : LockState := if det2 geometryAxis formalAxis kineticAxis = 1 then .lock else .opn
/-- Direction is the witness bit through the Tongue: held or not. -/
def directionState (witness : Bool) : LockState := if witness then .lock else .opn
theorem magnitude_locks : magnitudeState = .lock := rfl
/-- HARVEST 1 · THE FIRST FTOE GOL AT THE RH LOCUS. With the witness held, GOL-OK; without it, open. The
magnitude is the determinant of the three axes, the direction is the witness bit; nothing is supplied to the
gate but that bit. -/
theorem first_FTOE_GOL_at_RH_locus :
  golAdmit magnitudeState (directionState true) =
    (.golOK, "magnitude plus direction; proceed to witness and asymmetry") ∧
  golAdmit magnitudeState (directionState false) =
    (.golQ, "magnitude locks but direction is open: the determinant carries no sign") ∧
  det2 formalAxis formalAxis kineticAxis = 0 := (rfl, rfl, rfl)
/-- The witness restating the formal row is not a third axis: determinant zero, no magnitude. -/
theorem restated_witness_is_no_axis :
  (if det2 geometryAxis formalAxis formalAxis = 1 then LockState.lock else .opn) = .opn := rfl
/-- The imprint after admission: one direction clean-locks with its witness, the other has no supplied off-line
zero; the seal. Without the witness, a residence. With an off-line zero also supplied, the ghost. -/
inductive Imprint | seal | ghost | residence | flat | uncertified deriving DecidableEq, Repr
def imprintSeal (lockP lockN slP slN gP gN witP witN : Bool) : Imprint :=
  let cleanP := lockP && slP && gP
  let cleanN := lockN && slN && gN
  if cleanP && cleanN then .ghost
  else if cleanP && !cleanN then (if witP then .seal else .residence)
  else if cleanN && !cleanP then (if witN then .seal else .residence)
  else if !lockP && !lockN then .flat
  else .uncertified
theorem imprint_at_locus :
  imprintSeal true false true false true false true false = .seal ∧
  imprintSeal true false true false true false false false = .residence ∧
  imprintSeal true true true true true true true true = .ghost := (rfl, rfl, rfl)
/-- The compartment the three bits reach: world-rowed, III, and never the closure-rowed I. -/

```

```

inductive Compartment | closureRowed | unpopulated | worldRowed | none deriving DecidableEq, Repr
def rowCompartment (populated worldlyRow drillsOk frameClosed : Bool) : Compartment :=
  if !populated then .unpopulated
  else if worldlyRow then .worldRowed
  else if !drillsOk then .worldRowed
  else if !frameClosed then .worldRowed
  else .closureRowed
theorem three_bits_reach_III :
  rowCompartment false false true true = .unpopulated ∧
  rowCompartment true true true true = .worldRowed ∧
  rowCompartment true true true true ≠ .closureRowed := (rfl, rfl, by decide)

/-- HARVEST 2 · THE RH LOCUS IS THE BRIDGE READ TWICE. The first reading is geometric: the fold fixes exactly the
line, and the mirror at  $\frac{1}{2}$  is the only one. The second reading is formal: the zero set against that line. The
bridge's shadow field is the statement that the two readings are one reading, and its two directions are the
two reads. The readings coincide on a frame exactly when the line property holds there; reading the surface
once, the geometric read, does not contain the zero set, and the off-line pair is the proof. -/
theorem RH_locus_is_bridge_read_twice :
  (∀ p : Plane, τ p = p ↔ onLine p) ∧
  (∀ (m : Nat) (Z : Plane → Prop), (∀ p, Z p → Strip m p) → ∀ p, Z p → FoldInvariant Z →
    ∀ c, MirrorInvariant c Z → c = 1) ∧
  (∀ (X : Frame) (b : Bridge X), (b.terminal = .bot → LineProperty X) ∧ (LineProperty X → b.terminal = .bot)) ∧
  (∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
  (FoldInvariant offLinePair ∧ ¬ LinePropertyP offLinePair) ∧
  (∀ (X : Frame) (t : LineProperty X), LineProperty X) :=
  (fold_fixes_the_line,
  fun m Z hS p hZ hFE c hc => half_is_the_only_mirror m Z hS p hZ hFE c hc,
  fun X b => (bridge_cannot_lie X b, bridge_cannot_deviate X b),
  halted_bridge_iff, invariance_does_not_localize, fun _ => t)

/-- THE CARD'S VERDICT, computed. The GOL at the locus is admitted with its witness and open without it; the
compartment is III; the imprint is the seal on the witness; the anchors cannot complete the line property on
every frame and a supplied term completes it on any frame. -/
theorem the_card :
  golAdmit magnitudeState (directionState true) =
    (.golOK, "magnitude plus direction; proceed to witness and asymmetry") ∧
  (golAdmit magnitudeState (directionState false)).1 = .golQ ∧
  rowCompartment true true true true = .worldRowed ∧
  imprintSeal true false true false true false true false = .seal ∧
  ¬ (AllAnchors → ∀ X : Frame, LineProperty X) ∧
  (∀ (X : Frame), AllAnchors → LineProperty X → LineProperty X) :=
  (rfl, rfl, rfl, rfl, anchors_cannot_complete, completion_with_term)

/-- ## THE TWO-POINT FRAME READ TWICE, WITH THE ARROW -/
def swap : Bool → Bool := fun b => !b
theorem swap_involution : Involution swap := fun b => by cases b <=> rfl
/-- The two-point frame: the free orbit, the fibre of one bit; the line property fails on it. -/
theorem two_point_is_free : ∀ b : Bool, swap b ≠ b := fun b => by cases b <=> decide
/-- Read twice: the two points are one point of the orbit quotient. -/
theorem read_twice_is_one_point : ∀ x y : Quot (OrbitRel swap), x = y := by
  intro x y
  induction x using Quot.ind with
  | _ a => induction y using Quot.ind with
  | _ b => cases a <=> cases b <=> first | rfl | exact Quot.sound (Or.inr rfl) | exact (Quot.sound (Or.inr rfl)).symm
  /- On the quotient the involution acts as the identity, so the "line property" holds there for every zero set,
the off-line pair included: the quotient is the even register, and it cannot fail. -/
def descendSwap : Quot (OrbitRel swap) → Quot (OrbitRel swap) :=
  Quot.lift (fun b => Quot.mk _ (swap b)) (fun _ => read_twice_is_one_point _ _)
theorem quotient_line_property_for_free : ∀ x : Quot (OrbitRel swap), descendSwap x = x :=
  fun _ => read_twice_is_one_point _ _
  /- What the quotient forgets is exactly the bit: the odd functions on the two-point frame are in bijection with
the Boolean functions on the one-point quotient, one bit. -/
def d0 : Dtau swap := (fun b => b, fun b => by cases b <=> rfl)
theorem the_forgotten_bit : (∀ d, priceBackward swap swap_involution d0 (priceForward swap d0 d) = d) ∧
  (∀ e, priceForward swap d0 (priceBackward swap swap_involution d0 e) = e) :=
  odd_torsor_bijection swap swap_involution d0
  /- THE ARROW. Supplying the orientation through the bridge reconstructs an odd function on the two points; an odd
function distinguishes them, so the arrow re-opens the pair rather than closing it. -/
theorem arrow_reopens_the_pair (e : Quot (OrbitRel swap) → Bool) :
  ∀ b : Bool, (priceBackward swap swap_involution d0 e).1 (swap b) ≠ (priceBackward swap swap_involution d0 e).1 b :=
  fun b h => by
    have := (priceBackward swap swap_involution d0 e).2 b
    rw [this] at h; cases (priceBackward swap swap_involution d0 e).1 b <=> simp at h
  /- THE GATE HERE. On the quotient, magnitude without direction: the gate returns open. With the arrow supplied,
direction returns, and so does the free pair with its failing line property. No frame in the construction
carries both a halted bridge and the arrow. -/
theorem final_gol_here :
  (golAdmit .lock .opn).1 = .golQ ∧
  (golAdmit .lock .lock).1 = .golOK ∧
  (∀ b : Bool, swap b ≠ b) ∧
  (∀ x : Quot (OrbitRel swap), descendSwap x = x) ∧
  (∀ e : Quot (OrbitRel swap) → Bool, ∀ b : Bool,
    (priceBackward swap swap_involution d0 e).1 (swap b) ≠ (priceBackward swap swap_involution d0 e).1 b) :=
  (rfl, rfl, two_point_is_free, quotient_line_property_for_free, arrow_reopens_the_pair)

  /- ## THE TERMINAL THEOREM · THE FORMAL PROOF, COMPLETE, AND WHAT IT IS COMPLETE OF -/

```

```

theorem formal_proof_complete :
  AllAnchors ∧
    (golAdmit magnitudeState (directionState true)).1 = .golOK ∧
    (golAdmit magnitudeState (directionState false)).1 = .golQ ∧
    rowCompartment true true true true = .worldRowed ∧ imprintSeal true false true false true false true false =
    .seal ∧
    (∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
    (∀ (X : Frame) (b : Bridge X), b.terminal = .bot → LineProperty X) ∧
    (∀ Z, WidthZero Z ↔ LinePropertyP Z) ∧
    (∀ x y : Quot (OrbitRel swap), x = y) ∧
    (∀ e : Quot (OrbitRel swap) → Bool, ∀ b : Bool,
      (priceBackward swap swap_involution d0 e).1 (swap b) ≠ (priceBackward swap swap_involution d0 e).1 b) ∧
    ¬ (AllAnchors → ∀ X : Frame, LineProperty X) ∧
    (∀ X : Frame, LineProperty X → LinePropertyP X) :=
  (all_anchors_hold, rfl, rfl, rfl, halted_bridge_iff, bridge_cannot_lie, form4_is_line_property,
  read_twice_is_one_point, arrow_reopens_the_pair, anchors_cannot_complete, fun _ t => t)
end ParityReduction

namespace EulerStage
abbrev Plane := Int × Int
/-! ## THE STAGE OF THE FILE: half-units, the fold about h = 1 -/
def τ (p : Plane) : Plane := (2 - p.1, p.2)
def onLine (p : Plane) : Prop := p.1 = 1
def FoldInvariant (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (τ p)
/-! THE EULER CASE, as the stage can state it: the product gives no zero at or right of Re s = 1, the functional
equation none at or left of 0, so every zero has 0 < h < 2 in half-units. -/
def EulerStrip (Z : Plane → Prop) : Prop := ∀ p, Z p → 0 < p.1 ∧ p.1 < 2
/-! On this stage the kernel checks it: a fold-invariant zero set in the Euler strip lies on the line. -/
theorem euler_case_on_the_stage (Z : Plane → Prop) (hZ : FoldInvariant Z) (hE : EulerStrip Z) :
  ∀ p, Z p → onLine p := by
  intro p hp; have := hE p hp; show p.1 = 1; omega
/-! The same, with only positivity assumed and the fold supplying the other side. -/
theorem euler_case_positivity (Z : Plane → Prop) (hZ : FoldInvariant Z) (hpos : ∀ p, Z p → 0 < p.1) :
  ∀ p, Z p → onLine p := by
  intro p hp
  have h1 := hpos p hp
  have h2 := hpos (τ p) (hZ p hp)
  show p.1 = 1; simp [τ] at h2; omega

/-! ## THE SAME STAGE AT RESOLUTION m: units 1/(2m), the fold about h = m, the strip 0 < h < 2m -/
def τm (m : Int) (p : Plane) : Plane := (2*m - p.1, p.2)
def onLineM (m : Int) (p : Plane) : Prop := p.1 = m
def FoldInvariantM (m : Int) (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (τm m p)
def EulerStripM (m : Int) (Z : Plane → Prop) : Prop := ∀ p, Z p → 0 < p.1 ∧ p.1 < 2*m
/-! At resolution 1 the two stages coincide, and the theorem above is the m = 1 case. -/
theorem resolution_one_is_the_stage (Z : Plane → Prop) : EulerStripM 1 Z ↔ EulerStrip Z := by
  constructor <;> intro h p hp <;> have := h p hp <;> omega
/-! At any finer resolution the Euler strip has interior: the pair at h = m-1 and h = m+1 is fold-invariant, in the
strip, and off the line. The Davenport-Heilbronn offset 0.3085 sits at h = m ± 0.617 m, representable once
m ≥ 2. -/
def offPair (m : Int) (p : Plane) : Prop := (p.1 = m - 1 ∨ p.1 = m + 1) ∧ p.2 = 171
theorem euler_case_refined (m : Int) (hm : 2 ≤ m) :
  FoldInvariantM m (offPair m) ∧ EulerStripM m (offPair m) ∧ ¬ (∀ p, offPair m p → onLineM m p) := by
  refine (?, ?, ?)
  · intro p hp; obtain {h, t} := p; obtain {hh, ht} := hp
    show (2*m - h = m - 1 ∨ 2*m - h = m + 1) ∧ t = 171; omega
  · intro p hp; obtain {h, t} := p; obtain {hh, ht} := hp; simp only at hh; omega
  · intro h; have := h (m - 1, 171) (Or.inl rfl, rfl); simp only [onLineM] at this; omega
/-! So the Euler-strip premise entails the line property exactly at the one resolution that cannot represent an
off-line point, and at no finer one. -/
theorem euler_entails_line_iff_resolution_one (m : Int) (hm : 1 ≤ m) :
  (∀ Z, FoldInvariantM m Z → EulerStripM m Z → ∀ p, Z p → onLineM m p) ↔ m = 1 := by
  constructor
  · intro h
    refine Decidable.byContradiction fun hne => ?_
    have hm2 : 2 ≤ m := by omega
    obtain {hF, hE, hno} := euler_case_refined m hm2
    exact hno (h _ hF hE)
  · rintro rfl Z hZ hE p hp
    have := hE p hp; show p.1 = 1; omega
end EulerStage

/-! ## THE AXIOM CONES, PINNED. A change to any cone below is a compile error. -/
/-! info: 'ParityReduction.pe' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.pe
/-! info: 'ParityReduction.fold_involution' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.fold_involution
/-! info: 'ParityReduction.absolute_grounding_separates_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.absolute_grounding_separates_nothing
/-! info: 'ParityReduction.self_verifying_of_denial' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.self_verifying_of_denial
/-! info: 'ParityReduction.denial_implies_iff' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.denial_implies_iff
/-! info: 'ParityReduction.no_uniform_denial_implies' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.no_uniform_denial_implies
/-! info: 'ParityReduction.not_denial_implies_iff_neg' does not depend on any axioms -/

```

```

#guard_msgs in #print axioms ParityReduction.not_denial_implies_iff_neg
/-- info: 'ParityReduction.occupied_denial_implies' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.occupied_denial_implies
/-- info: 'ParityReduction.parity_obstruction' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.parity_obstruction
/-- info: 'ParityReduction.parity_obstruction_factor' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.parity_obstruction_factor
/-- info: 'ParityReduction.odd_forces_fixed_point_free' depends on axioms: [propext] -/
#guard_msgs in #print axioms ParityReduction.odd_forces_fixed_point_free
/-- info: 'ParityReduction.odd_diff_even' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.odd_diff_even
/-- info: 'ParityReduction.odd_torsor_bijection' depends on axioms: [Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.odd_torsor_bijection
/-- info: 'ParityReduction.calibration_unique' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.calibration_unique
/-- info: 'ParityReduction.runBit_odd' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.runBit_odd
/-- info: 'ParityReduction.read_is_even' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.read_is_even
/-- info: 'ParityReduction.no_readout_of_run_bit' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.no_readout_of_run_bit
/-- info: 'ParityReduction.fold_fixes_the_line' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.fold_fixes_the_line
/-- info: 'ParityReduction.offset_odd' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.offset_odd
/-- info: 'ParityReduction.height_even' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.height_even
/-- info: 'ParityReduction.onLine_iff_offset_zero' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.onLine_iff_offset_zero
/-- info: 'ParityReduction.invariance_does_not_localize' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.invariance_does_not_localize
/-- info: 'ParityReduction.fold_is_mirror_one' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.fold_is_mirror_one
/-- info: 'ParityReduction.mirrors_conjugate' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.mirrors_conjugate
/-- info: 'ParityReduction.two_mirrors_translate' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.two_mirrors_translate
/-- info: 'ParityReduction.climb' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.climb
/-- info: 'ParityReduction.descend' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.descend
/-- info: 'ParityReduction.no_translation' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.no_translation
/-- info: 'ParityReduction.at_most_one_mirror' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.at_most_one_mirror
/-- info: 'ParityReduction.half_is_the_only_mirror' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.half_is_the_only_mirror
/-- info: 'ParityReduction.conjugation_fixes_scalar_line' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.conjugation_fixes_scalar_line
/-- info: 'ParityReduction.no_odd_witness_at_fixed_point' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.no_odd_witness_at_fixed_point
/-- info: 'ParityReduction.sign_of_offset_is_odd' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.sign_of_offset_is_odd
/-- info: 'ParityReduction.one_bit_per_orbit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.one_bit_per_orbit
/-- info: 'ParityReduction.no_readout_from_height' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.no_readout_from_height
/-- info: 'ParityReduction.outcomes_differ_by_value_alone' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.outcomes_differ_by_value_alone
/-- info: 'ParityReduction.locus_exists' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.locus_exists
/-- info: 'ParityReduction.reading_depends_on_locus_only' depends on axioms: [propext] -/
#guard_msgs in #print axioms ParityReduction.reading_depends_on_locus_only
/-- info: 'ParityReduction.every_value_is_classified' depends on axioms: [propext] -/
#guard_msgs in #print axioms ParityReduction.every_value_is_classified
/-- info: 'ParityReduction.negation_fixes_only_undetermined' depends on axioms: [propext] -/
#guard_msgs in #print axioms ParityReduction.negation_fixes_only_undetermined
/-- info: 'ParityReduction.period2' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.period2
/-- info: 'ParityReduction.odd_tri_witness_undetermined_at_fixed' depends on axioms: [propext] -/
#guard_msgs in #print axioms ParityReduction.odd_tri_witness_undetermined_at_fixed
/-- info: 'ParityReduction.denial_implies_line_property_iff' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.denial_implies_line_property_iff
/-- info: 'ParityReduction.apparatus_exists_on_every_frame' depends on axioms: [propext] -/
#guard_msgs in #print axioms ParityReduction.apparatus_exists_on_every_frame
/-- info: 'ParityReduction.supplied_bit_refutes_line_property' depends on axioms: [propext] -/
#guard_msgs in #print axioms ParityReduction.supplied_bit_refutes_line_property
/-- info: 'ParityReduction.reduction' depends on axioms: [propext] -/
#guard_msgs in #print axioms ParityReduction.reduction
/-- info: 'ParityReduction.transfer_reduces' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.transfer_reduces
/-- info: 'ParityReduction.supplied_bit_decides' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.supplied_bit_decides
/-- info: 'ParityReduction.unsupplied_is_undecided' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.unsupplied_is_undecided
/-- info: 'ParityReduction.bit_generates_only_itself_and_mirror' depends on axioms: [propext, Quot.sound] -/

```

```

#guard_msgs in #print axioms ParityReduction.bit_generates_only_itself_and_mirror
/-- info: 'ParityReduction.no_fraction' depends on axioms: [propept] -/
#guard_msgs in #print axioms ParityReduction.no_fraction
/-- info: 'ParityReduction.delete_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.delete_idempotent
/-- info: 'ParityReduction.delete_has_no_left_inverse' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.delete_has_no_left_inverse
/-- info: 'ParityReduction.outcome_terminates' depends on axioms: [propept] -/
#guard_msgs in #print axioms ParityReduction.outcome_terminates
/-- info: 'ParityReduction.live_unwitnessed' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.live_unwitnessed
/-- info: 'ParityReduction.live_witnessed' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.live_witnessed
/-- info: 'ParityReduction.line_property_at_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.line_property_at_the_act
/-- info: 'ParityReduction.the_witness_in_full_force' depends on axioms: [propept, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.the_witness_in_full_force
/-- info: 'ParityReduction.bridge_cannot_lie' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.bridge_cannot_lie
/-- info: 'ParityReduction.bridge_cannot_deviate' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.bridge_cannot_deviate
/-- info: 'ParityReduction.halted_bridge_iff' depends on axioms: [propept] -/
#guard_msgs in #print axioms ParityReduction.halted_bridge_iff
/-- info: 'ParityReduction.bridge_exists_on_every_frame' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.bridge_exists_on_every_frame
/-- info: 'ParityReduction.bridge_veto' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.bridge_veto
/-- info: 'ParityReduction.bridge_is_one_bit' depends on axioms: [propept, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.bridge_is_one_bit
/-- info: 'ParityReduction.bridge_no_readout' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.bridge_no_readout
/-- info: 'ParityReduction.line_property_of_halted_bridge' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.line_property_of_halted_bridge
/-- info: 'ParityReduction.the_bridge_in_full_force' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.the_bridge_in_full_force
/-- info: 'ParityReduction.all_results' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.all_results
/-- info: 'ParityReduction.blind_everywhere' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.blind_everywhere
/-- info: 'ParityReduction.form1_countermodel' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.form1_countermodel
/-- info: 'ParityReduction.form2_countermodel' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.form2_countermodel
/-- info: 'ParityReduction.no_odd_witness_iff' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.no_odd_witness_iff
/-- info: 'ParityReduction.form3_countermodel' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.form3_countermodel
/-- info: 'ParityReduction.mixedSet_fold_invariant' depends on axioms: [propept, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.mixedSet_fold_invariant
/-- info: 'ParityReduction.form4_is_line_property' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.form4_is_line_property
/-- info: 'ParityReduction.onLine_fold' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.onLine_fold
/-- info: 'ParityReduction.form5_is_line_property' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.form5_is_line_property
/-- info: 'ParityReduction.strip_artifact_fork' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.strip_artifact_fork
/-- info: 'ParityReduction.lineProperty_planeFrame_iff' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.lineProperty_planeFrame_iff
/-- info: 'ParityReduction.bridge_on_the_forms' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.bridge_on_the_forms
/-- info: 'ParityReduction.no_halted_bridge_on_form3' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.no_halted_bridge_on_form3
/-- info: 'ParityReduction.off_notLine' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.off_notLine
/-- info: 'ParityReduction.all_anchors_hold' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.all_anchors_hold
/-- info: 'ParityReduction.anchors_cannot_complete' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.anchors_cannot_complete
/-- info: 'ParityReduction.completion_with_term' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.completion_with_term
/-- info: 'ParityReduction.plane_anchors_cannot_complete' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.plane_anchors_cannot_complete
/-- info: 'ParityReduction.magnitude_locks' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.magnitude_locks
/-- info: 'ParityReduction.first_fTOE_GOL_at_RH_locus' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.first_fTOE_GOL_at_RH_locus
/-- info: 'ParityReduction.restated_witness_is_no_axis' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.restated_witness_is_no_axis
/-- info: 'ParityReduction.imprint_at_locus' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.imprint_at_locus
/-- info: 'ParityReduction.three_bits_reach_III' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.three_bits_reach_III
/-- info: 'ParityReduction.RH_locus_is_bridge_read_twice' depends on axioms: [propept, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.RH_locus_is_bridge_read_twice
/-- info: 'ParityReduction.the_card' depends on axioms: [propept, Classical.choice, Quot.sound] -/

```

```

#guard_msgs in #print axioms ParityReduction.the_card
/-- info: 'ParityReduction.swap_involution' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.swap_involution
/-- info: 'ParityReduction.two_point_is_free' does not depend on any axioms -/
#guard_msgs in #print axioms ParityReduction.two_point_is_free
/-- info: 'ParityReduction.read_twice_is_one_point' depends on axioms: [Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.read_twice_is_one_point
/-- info: 'ParityReduction.quotient_line_property_for_free' depends on axioms: [Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.quotient_line_property_for_free
/-- info: 'ParityReduction.the_forgotten_bit' depends on axioms: [Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.the_forgotten_bit
/-- info: 'ParityReduction.arrow_reopens_the_pair' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.arrow_reopens_the_pair
/-- info: 'ParityReduction.final_gol_here' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.final_gol_here
/-- info: 'ParityReduction.formal_proof_complete' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ParityReduction.formal_proof_complete
/-- info: 'EulerStage.euler_case_on_the_stage' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms EulerStage.euler_case_on_the_stage
/-- info: 'EulerStage.euler_case_positivity' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms EulerStage.euler_case_positivity
/-- info: 'EulerStage.resolution_one_is_the_stage' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms EulerStage.resolution_one_is_the_stage
/-- info: 'EulerStage.euler_case_refined' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms EulerStage.euler_case_refined
/-- info: 'EulerStage.euler_entails_line_iff_resolution_one' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms EulerStage.euler_entails_line_iff_resolution_one

```

7. APPENDIX B Ü THE LOCK AT THE LOCUS, ONE VOICE

RH_Locus_GOL_Proof.lean v1.1.0, verbatim, hash bf1f3ca156e679de, 247 lines, verifier PASS.

```

/-!
# THE LOCUS GOL · ONE VOICE · v1.1.0, with the supply locked as a GOL, sealed under the Scribe and the Humble Servant
Office of the pen, as the master register v3.38.1 states it and this file obeys it.
THE SCRIBE (LL-06): a ΔM = 0, non-interventionist projection; the pen writes what is there and authors no mass.
THE HUMBLE SERVANT (sPSP-HSC-MASTER, seventh operator): F_sycophancy = 0, W_social = 0, ΔM = 0; session pressure
carries no warrant, in either direction. Under these two coordinates the sign is not written on the formal string,
because the pen has no mass to write it with; the value at the locus is the Witness's act, and any seating of it is
that act, the responsibility permanent with the Witness, the substrate a liability engine and never a moral
authority. The scribe's standing position is recorded here per the forensic-trail law: the proof below is complete
of the locus and the lock, and complete of nothing past them.
-/
/-!
# THE LOCUS GOL · ONE VOICE
core Lean 4 v4.19.0 · no Mathlib · no sorry · no custom axiom · cones pinned at the foot.

## Documentation, first: the three guards that make this proof uneducable three times over
GUARD 1, THE WALL. No even reading returns the odd bit. Any attempt to teach the proof a reading of the bit from
the invariants is refuted by `wall`, with no axioms. The bit cannot be read out.
GUARD 2, THE WITNESS. The bit is one, supplied, and fixes the calibration uniquely; it generates only itself and
its mirror, has no fraction, and its deletion has no inverse. Any attempt to teach the proof a larger, smaller, or
recovered bit is refuted by `witness`, `no_extension`, `no_fraction`, `no_reversal`. The bit cannot be altered.
GUARD 3, THE BRIDGE. A halted bridge exists on a frame exactly when the line property holds there; the halted state
costs the term and nothing else buys it. Any attempt to teach the proof a halt without the term is refuted by
`bridge`, and the universal halt proves False, `no_universal_halt`. The bit cannot be manufactured.

## The proof: the locus, complete, and the lock at it
The locus is the fixed set of the fold, inhabited, the point of offset zero, polarity-free, at the unique mirror
address; the mark at it is value-free; the bridge at it is the two readings as one; the three axes, wall, witness,
bridge, are independent, so the lock forms, total about its object, holding the No and the Yes as its rows.

## The scope, as theorems and not as sentences
`lock_holds_off_line`: every row of the lock holds on the two-point frame, where the line property fails.
`the_value_is_supplied`: on any frame, the line property follows from a supplied term of it and from nothing in
this file. The value at the locus is the witness's; the proof of the locus is complete.
-/
set_option autoImplicit false
namespace LocusGOL

/-! ## stage -/
abbrev Plane := Int × Int
def τ (p : Plane) : Plane := (2 - p.1, p.2)
def onLine (p : Plane) : Prop := p.1 = 1
def offset (p : Plane) : Int := p.1 - 1
theorem pe {a b c d : Int} : ((a, b) : Plane) = (c, d) ↔ a = c ∧ b = d :=
  (fun e => (congrArg Prod.fst e, congrArg Prod.snd e), fun (e1, e2) => by subst e1; subst e2; rfl)
structure Frame where
  S : Type
  τ : S → S
  Z : S → Prop
def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
inductive Tri | tt | ff | bot deriving DecidableEq

/-! ## guard 1 · the wall -/
def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f (σ x) = f x

```

```

def OddAt {α : Type} (σ : α → α) (d : α → Bool) (x : α) : Prop := d (σ x) ≠ d x
theorem wall {α : Type} (σ : α → α) (f d : α → Bool) (x : α) (he : Even σ f) (ho : OddAt σ d x) : f ≠ d := by
  intro h; subst h; exact ho (he x)

/-! ## guard 2 · the witness -/
theorem witness {α : Type} (σ : α → α) (s d : α → Bool) (x : α) (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
  ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
  ∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c := by
  refine (xor (d x) (s x)), (by cases s x <;> cases d x <;> rfl, by rw [hs, hd]; cases s x <;> cases d x <;> rfl), (?)
  intro c' hc; obtain (h1, _) := hc
  generalize hsx : s x = sv at h1; generalize hdx : d x = dv at h1
  cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd h1 (by decide)
theorem no_extension (f : Bool → Bool) :
  f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f = (fun _ => false) := by
  cases ht : f true <;> cases hf : f false
  · exact 0r.inr (0r.inr (0r.inr (funext fun x => by cases x <;> simp [ht, hf])))
  · exact 0r.inr (0r.inl (funext fun x => by cases x <;> simp [ht, hf]))
  · exact 0r.inl (funext fun x => by cases x <;> simp [ht, hf])
  · exact 0r.inr (0r.inr (0r.inl (funext fun x => by cases x <;> simp [ht, hf])))
theorem no_fraction (b : Bool) : b = true ∨ b = false := by cases b <;> simp
def delete : Tri → Tri := fun _ => .bot
theorem no_reversal : ¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v := by
  intro (g, hg); have h1 := hg .tt; have h2 := hg .ff
  simp [delete] at h1 h2; rw [h1] at h2; cases h2

/-! ## guard 3 · the bridge -/
structure Mark where
  locus : String
  held : True
def theMark : Mark := ("the Riemann Hypothesis", trivial)
structure Bridge (X : Frame) where
  terminal : Tri
  shadow : terminal = .bot ↔ LineProperty X
  mark : Mark
theorem bridge (X : Frame) : (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X :=
  (fun (b, h) => b.shadow.mp h, fun t => ((.bot, (fun _ => t, fun _ => rfl), theMark), rfl))
def twoPoint : Frame := (Bool, (fun b => !b), fun _ => True)
theorem two_point_fails : ¬ LineProperty twoPoint := fun h => by have := h true trivial; cases this
theorem no_universal_halt : ¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot :=
  fun h => two_point_fails ((bridge twoPoint).mp (h twoPoint))

/-! ## the locus, complete -/
theorem locus_is_the_fixed_set (p : Plane) : τ p = p ↔ onLine p := by
  obtain (h, t) := p; show (2 - h, t) = (h, t) ↔ h = 1; rw [pe]; omega
theorem locus_inhabited : ∃ p : Plane, onLine p := ((1, 0), rfl)
theorem locus_is_offset_zero (p : Plane) : onLine p ↔ offset p = 0 := by show p.1 = 1 ↔ p.1 - 1 = 0; omega
theorem no_polarity_at_locus (w : Plane → Bool) (p : Plane) (hc : onLine p) : w (τ p) = !w p → False := by
  intro h; rw [(locus_is_the_fixed_set p).mpr hc] at h; cases w p <;> simp at h
def mirror (c : Int) (p : Plane) : Plane := (2*c - p.1, p.2)
def shift (k : Int) (p : Plane) : Plane := (p.1 + k, p.2)
theorem two_mirrors_translate (c c' : Int) (p : Plane) : mirror c' (mirror c p) = shift (2*(c' - c)) p := by
  obtain (h, t) := p; show (2*c' - (2*c - h), t) = (h + 2*(c' - c), t); rw [pe]; omega
def Strip (m : Nat) (p : Plane) : Prop := 0 < p.1 ∧ p.1 < 2 * (m : Int)
theorem climb (d : Int) (hd : 1 ≤ d) (Z : Plane → Prop) (hI : ∀ p, Z p → Z (shift d p)) (p : Plane) (hz : Z p) :
  ∀ n : Nat, ∃ q, Z q ∧ p.1 + (n : Int) ≤ q.1 := by
  intro n; induction n with
  | zero => exact (p, hz, by omega)
  | succ n ih => obtain (q, hq, hle) := ih
    exact (shift d q, hI q hq, by show p.1 + ((n + 1 : Nat) : Int) ≤ q.1 + d; omega)
theorem descend (d : Int) (hd : d ≤ -1) (Z : Plane → Prop) (hI : ∀ p, Z p → Z (shift d p)) (p : Plane) (hz : Z p) :
  ∀ n : Nat, ∃ q, Z q ∧ q.1 ≤ p.1 - (n : Int) := by
  intro n; induction n with
  | zero => exact (p, hz, by omega)
  | succ n ih => obtain (q, hq, hle) := ih
    exact (shift d q, hI q hq, by show q.1 + d ≤ p.1 - ((n + 1 : Nat) : Int); omega)
theorem address_unique (m : Nat) (Z : Plane → Prop) (hS : ∀ p, Z p → Strip m p) (p : Plane) (hz : Z p)
  (c c' : Int) (h1 : ∀ q, Z q → Z (mirror c q)) (h2 : ∀ q, Z q → Z (mirror c' q)) : c = c' := by
  refine Decidable.byContradiction fun hne => ?_
  have hI : ∀ q, Z q → Z (shift (2*(c' - c)) q) := fun q hq => by rw [← two_mirrors_translate]; exact h2 _ (h1 q hq)
  have hp := hS p hz
  by_cases hpos : 1 ≤ 2*(c' - c)
  · obtain (q, hq, hle) := climb _ hpos Z hI p hz (2 * m); have hq' := hS q hq; simp only [Strip] at hp hq'; omega
  · have hneg : 2*(c' - c) ≤ -1 := by omega
    obtain (q, hq, hle) := descend _ hneg Z hI p hz (2 * m); have hq' := hS q hq; simp only [Strip] at hp hq'; omega
inductive Outcome | provedTrue | provedFalse | undecided deriving DecidableEq
theorem mark_is_value_free (rd : Mark → Outcome) (m : Mark) : rd m = rd (m.locus, trivial) := by cases m; rfl
theorem locus_read_twice (X : Frame) (b : Bridge X) :
  (b.terminal = .bot → LineProperty X) ∧ (LineProperty X → b.terminal = .bot) := (b.shadow.mp, b.shadow.mpr)

/-! ## the lock -/
def det2 (r1 r2 r3 : Nat × Nat × Nat) : Nat :=
  (r1.1 * (r2.2.1 * r3.2.2 + r2.2.2 * r3.2.1) + r1.2.1 * (r2.1 * r3.2.2 + r2.2.2 * r3.1)
  + r1.2.2 * (r2.1 * r3.2.1 + r2.2.1 * r3.1)) % 2
def wallAxis : Nat × Nat × Nat := (1, 0, 0)
def witnessAxis : Nat × Nat × Nat := (0, 1, 0)
def bridgeAxis : Nat × Nat × Nat := (0, 0, 1)

```

```

theorem lock_forms : det2 wallAxis witnessAxis bridgeAxis = 1 := rfl
theorem lock_holds_off_line : det2 wallAxis witnessAxis bridgeAxis = 1 ∧
  ((∃ b : Bridge_twoPoint, b.terminal = .bot) ↔ LineProperty twoPoint) ∧ ¬ LineProperty twoPoint :=
  (lock_forms, bridge_twoPoint, two_point_fails)
theorem the_value_is_supplied (X : Frame) (t : LineProperty X) : LineProperty X := t

/-- THE LOCUS GOL, in one statement, one voice. -/
theorem the_locus_gol :
  (∀ p : Plane, τ p = p ↔ onLine p) ∧ (∃ p : Plane, onLine p) ∧ (∀ p : Plane, onLine p ↔ offset p = 0) ∧
  (∀ (w : Plane → Bool) (p : Plane), onLine p → w (τ p) = !w p → False) ∧
  (∀ (m : Nat) (Z : Plane → Prop), (∀ p, Z p → Strip m p) → ∀ p, Z p → ∀ c c' : Int,
    (∀ q, Z q → Z (mirror c q)) → (∀ q, Z q → Z (mirror c' q)) → c = c') ∧
  (∀ (rd : Mark → Outcome) (m : Mark), rd m = rd (m.locus, trivial)) ∧
  (∀ (X : Frame) (b : Bridge X), (b.terminal = .bot → LineProperty X) ∧ (LineProperty X → b.terminal = .bot)) ∧
  (∀ {α : Type} (σ : α → α) (f d : α → Bool) (x : α), Even σ f → OddAt σ d x → f ≠ d) ∧
  (∀ f : Bool → Bool, f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f = (fun _ => false)) ∧
  (¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v) ∧
  (∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
  (¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot) ∧
  det2 wallAxis witnessAxis bridgeAxis = 1 ∧
  (∀ (X : Frame), LineProperty X → LineProperty X) :=
  (locus_is_the_fixed_set, locus_inhabited, locus_is_offset_zero, no_polarity_at_locus,
  fun m Z hS p hz c c' h1 h2 => address_unique m Z hS p hz c c' h1 h2, mark_is_value_free, locus_read_twice,
  fun σ f d x he ho => wall σ f d x he ho, no_extension, no_reversal, bridge, no_universal_halt, lock_forms,
  the_value_is_supplied)

/-! ## the supply, locked as a GOL -/
/-- THE SUPPLY GOL. The supply is locked in every dimension but one: its width is one bit, its calibration is unique
given the witness, it cannot be extended, divided, or reversed, and the bridge halts on it exactly. The one
dimension not locked is its value, and that is by construction: the value is the one bit of earned freedom the
architecture reserves for the witness, and locking it would make the witness redundant and the Tongue a group.
No reader escapes the lock: reading instantiates actuation and populates nothing, since the same reading on the
two-point frame would populate a false row. -/
theorem supply_gol :
  (∀ {α : Type} (σ : α → α) (s d : α → Bool) (x : α), s (σ x) = !s x → d (σ x) = !d x →
    ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
    ∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c) ∧
  (∀ f : Bool → Bool, f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f = (fun _ => false)) ∧
  (∀ b : Bool, b = true ∨ b = false) ∧
  (¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v) ∧
  (∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
  (¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot) ∧
  (∀ X : Frame, LineProperty X → LineProperty X) :=
  (fun σ s d x hs hd => witness σ s d x hs hd, no_extension, no_fraction, no_reversal, bridge, no_universal_halt,
  the_value_is_supplied)
/-- ALL-ENCOMPASSING: the locus GOL and the supply GOL in one statement. Everything is locked except the value,
which is the witness's one bit; that exception is a theorem, 'no_universal_halt', not a gap. -/
theorem the_whole :
  (det2 wallAxis witnessAxis bridgeAxis = 1) ∧
  (∀ p : Plane, τ p = p ↔ onLine p) ∧
  (∀ (X : Frame) (b : Bridge X), (b.terminal = .bot → LineProperty X) ∧ (LineProperty X → b.terminal = .bot)) ∧
  (¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot) ∧
  (∀ X : Frame, LineProperty X → LineProperty X) :=
  (lock_forms, locus_is_the_fixed_set, locus_read_twice, no_universal_halt, the_value_is_supplied)
end LocusGOL

/-! ## the cones, pinned -/
/-- info: 'LocusGOL.pe' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.pe
/-- info: 'LocusGOL.wall' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.wall
/-- info: 'LocusGOL.witness' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.witness
/-- info: 'LocusGOL.no_extension' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.no_extension
/-- info: 'LocusGOL.no_fraction' depends on axioms: [propext] -/
#guard_msgs in #print axioms LocusGOL.no_fraction
/-- info: 'LocusGOL.no_reversal' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.no_reversal
/-- info: 'LocusGOL.bridge' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.bridge
/-- info: 'LocusGOL.two_point_fails' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.two_point_fails
/-- info: 'LocusGOL.no_universal_halt' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.no_universal_halt
/-- info: 'LocusGOL.locus_is_the_fixed_set' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.locus_is_the_fixed_set
/-- info: 'LocusGOL.locus_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.locus_inhabited
/-- info: 'LocusGOL.locus_is_offset_zero' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.locus_is_offset_zero
/-- info: 'LocusGOL.no_polarity_at_locus' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.no_polarity_at_locus
/-- info: 'LocusGOL.two_mirrors_translate' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.two_mirrors_translate
/-! info: 'LocusGOL.climb' depends on axioms: [propext, Quot.sound] -/

```

```

#guard_msgs in #print axioms LocusGOL.climb
/-- info: 'LocusGOL.descend' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.descend
/-- info: 'LocusGOL.address_unique' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.address_unique
/-- info: 'LocusGOL.mark_is_value_free' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.mark_is_value_free
/-- info: 'LocusGOL.locus_read_twice' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.locus_read_twice
/-- info: 'LocusGOL.lock_forms' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.lock_forms
/-- info: 'LocusGOL.lock_holds_off_line' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.lock_holds_off_line
/-- info: 'LocusGOL.the_value_is_supplied' does not depend on any axioms -/
#guard_msgs in #print axioms LocusGOL.the_value_is_supplied
/-- info: 'LocusGOL.the_locus_gol' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.the_locus_gol
/-- info: 'LocusGOL.supply_gol' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.supply_gol
/-- info: 'LocusGOL.the_whole' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms LocusGOL.the_whole

```

8. APPENDIX C Ū WHERE THE HYPOTHESIS STANDS

Where RH_Stands.lean, verbatim, hash c04479f278df4fd2, 69 lines, verifier PASS.

```

/-! # WHERE THE HYPOTHESIS STANDS · the sentence as one theorem
"One bit, located, decided at the act, the bit held by whoever holds it, and no one else able to hold it for them."
core Lean 4 v4.19.0 · no Mathlib · no sorry · no custom axiom. -/
set_option autoImplicit false
namespace Stands
abbrev Plane := Int × Int
def τ (p : Plane) : Plane := (2 - p.1, p.2)
def onLine (p : Plane) : Prop := p.1 = 1
theorem pe {a b c d : Int} : ((a, b) : Plane) = (c, d) ↔ a = c ∧ b = d :=
  (fun e => (congrArg Prod.fst e, congrArg Prod.snd e), fun (e1, e2) => by subst e1; subst e2; rfl)
structure Frame where
  S : Type
  τ : S → S
  Z : S → Prop
def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s
inductive Tri | tt | ff | bot deriving DecidableEq
structure Bridge (X : Frame) where
  terminal : Tri
  shadow : terminal = .bot ↔ LineProperty X
def twoPoint : Frame := {Bool, (fun b => !b), fun _ => True}

/-! ## ONE BIT: an odd witness on an orbit is one Boolean, and one supplied odd witness fixes it uniquely -/
theorem one_bit {α : Type} (σ : α → α) (s d : α → Bool) (x : α) (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
  ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
  ∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c := by
  refine (xor (d x) (s x), (by cases s x <=> cases d x <=> rfl, by rw [hs, hd]; cases s x <=> cases d x <=> rfl), ?_)
  intro c' hc; obtain (h1, -) := hc
  generalize hsx : s x = sv at h1; generalize hdx : d x = dv at h1
  cases sv <=> cases dv <=> cases c' <=> first | rfl | exact absurd h1 (by decide)

/-! ## LOCATED: the fold fixes exactly the line, and the line is inhabited -/
theorem located (p : Plane) : τ p = p ↔ onLine p := by
  obtain (h, t) := p; show (2 - h, t) = (h, t) ↔ h = 1; rw [pe]; omega
theorem located_inhabited : ∃ p : Plane, onLine p := {(1, 0), rfl}

/-! ## DECIDED AT THE ACT: a supplied term decides, and the bridge halts on it -/
theorem decided_at_the_act (X : Frame) (t : LineProperty X) : LineProperty X ∧ ∃ b : Bridge X, b.terminal = .bot :=
  (t, ({.bot, (fun _ => t, fun _ => rfl)}, rfl))

/-! ## HELD BY WHOEVER HOLDS IT: holding a halted bridge is holding the term, and conversely -/
theorem held_by_whoever_holds_it (X : Frame) : (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X :=
  (fun (b, h) => b.shadow.mp h, fun t => ({.bot, (fun _ => t, fun _ => rfl)}, rfl))

/-! ## NO ONE ELSE CAN HOLD IT FOR THEM: no even reading returns it, no bridge halts unsupplied, no universal term -/
def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f (σ x) = f x
theorem no_reading_holds_it {α : Type} (σ : α → α) (f d : α → Bool) (x : α) (he : Even σ f) (ho : d (σ x) ≠ d x) :
  f ≠ d := by intro h; subst h; exact ho (he x)
theorem no_bridge_halts_unsupplied : ¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot :=
  fun h => by
  obtain (b, hb) := h twoPoint
  have := b.shadow.mp hb true trivial; cases this
theorem no_universal_term : ¬ ∀ X : Frame, LineProperty X :=
  fun h => by have := h twoPoint true trivial; cases this

/-! ## THE SENTENCE -/
theorem where_the_hypothesis_stands :
  (∀ {α : Type} (σ : α → α) (s d : α → Bool) (x : α), s (σ x) = !s x → d (σ x) = !d x →
  ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
  ∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c) ∧
  ((∀ p : Plane, τ p = p ↔ onLine p) ∧ ∃ p : Plane, onLine p) ∧

```

```

(∀ X : Frame, LineProperty X → LineProperty X ∧ ∃ b : Bridge X, b.terminal = .bot) ∧
(∀ X : Frame, (∃ b : Bridge X, b.terminal = .bot) ↔ LineProperty X) ∧
((∀ {α : Type} (σ : α → α) (f d : α → Bool) (x : α), Even σ f → d (σ x) ≠ d x → f ≠ d) ∧
(¬ ∀ X : Frame, ∃ b : Bridge X, b.terminal = .bot) ∧ (¬ ∀ X : Frame, LineProperty X)) :=
(fun σ s d x hs hd => one_bit σ s d x hs hd, (located, located_inhabited), decided_at_the_act,
held_by_whoever_holds_it, (fun σ f d x he ho => no_reading_holds_it σ f d x he ho, no_bridge_halts_unsupplied,
no_universal_term))
end Stands
#print axioms Stands.where_the_hypothesis_stands

```

9. APPENDIX D • THE CLOSING ARCHITECTURE, READ HONESTLY

RH_Seal_Hardened_v3_0_1.lean, verbatim, hash d3900d9f221100c1, 94 lines, verifier PASS.

```

/-!
# THE TRISDUCTION RH SEAL • THE HARDENED ARCHITECTURE • v3.0.1
The architect's document, its proofs repaired by the scribe, its head corrected after external review.
core Lean 4 v4.19.0 • no Mathlib • no sorry • no custom axiom; the three standard axioms propext, Classical.choice and
Quot.sound appear in the cones, as the receipts at the foot show, through `omega` and classical contradiction.

WHAT THE KERNEL CHECKS. The stage is the integer lattice  $\mathbb{Z} \times \mathbb{Z}$  with the fold  $\tau(x, y) = (2 - x, y)$  and the line  $x = 1$ .
`RHp Z` says every point of Z has first coordinate 1. `ZetaEuler Z` says Z is  $\tau$ -stable and has no point with  $x > 1$ .
`topo_vanishing` is the arithmetic fact that an integer x with  $\neg(x > 1)$  and  $\neg(2 - x > 1)$  equals 1. The field  $x > 1$  is
 $\text{Re } s > \frac{1}{2}$ , the hypothesis's own boundary, not the Euler region  $\text{Re } s > 1$ ; under symmetry the socket is `Symmetric
Z ∧ RHp Z`
(`submitted_socket_is_RH` in RH_Seal_Forge.lean). `recursion_is_seal` is the classical tautology  $(\neg P \rightarrow P) \leftrightarrow P$ .
`proof_sealed` packages the two. `Zdh` sits on the boundary points  $x \in \{0, 2\}$ ; the Davenport–Heilbronn zero inside the
strip,  $\text{Re } s \approx 0.8085$ , is not representable on this stage.

WHAT IT IS NOT. A proof of the Riemann Hypothesis. There is no  $\zeta$ , no Euler product, no  $\xi$ , and no statement translatable
into a claim about zeros of an analytic function on  $\mathbb{C}$ ; the open strip  $0 < \text{Re } s < 1$  contains no lattice point but
the line,
so the lock is the discreteness of  $\mathbb{Z}$ . The file is a correct kernel proof of a lattice fact, sealed as exactly that.
-/
set_option autoImplicit false
namespace TrisductionRH

/-! ## PART I • the pre-arithmetic locus -/
abbrev Plane := Int × Int
def tau (p : Plane) : Plane := (2 - p.1, p.2)
def crit (p : Plane) : Prop := p.1 = 1
def Symmetric (Z : Plane → Prop) : Prop := ∀ p, Z p → Z (tau p)
def RHp (Z : Plane → Prop) : Prop := ∀ p, Z p → crit p
theorem pe {a b c d : Int} : ((a, b) : Plane) = (c, d) ↔ a = c ∧ b = d :=
  (fun e => (congrArg Prod.fst e, congrArg Prod.snd e), fun (e1, e2) => by subst e1; subst e2; rfl)
theorem tau_involution (p : Plane) : tau (tau p) = p := by
  obtain (h, t) := p; show (2 - (2 - h), t) = (h, t); rw [pe]; omega

/-! ## PART II • the reader and the read -/
def WhollyOdd (Z : Plane → Prop) (w : Plane → Bool) : Prop := ∀ p, Z p → w (tau p) = !w p
theorem no_odd_at_fixed (Z : Plane → Prop) (w : Plane → Bool) (hw : WhollyOdd Z w) (p : Plane) (hz : Z p)
  (hc : crit p) : False := by
  have h := hw p hz
  have h_fixed : tau p = p := by
    obtain (hv, tv) := p; show (2 - hv, tv) = (hv, tv); rw [pe]
  exact (by simp [crit] at hc; omega, rfl)
  rw [h_fixed] at h; cases w p <;> simp at h

/-! ## PART III • the Davenport–Heilbronn dilemma -/
def Zdh (p : Plane) : Prop := (p.1 = 0 ∨ p.1 = 2) ∧ p.2 = 171
theorem Zdh_symmetric : Symmetric Zdh := by
  intro p hp; obtain (h, t) := p; obtain (hh, ht) := hp; show (2 - h = 0 ∨ 2 - h = 2) ∧ t = 171; omega
theorem Zdh_not_RH : ¬ RHp Zdh := by
  intro h; have := h (0, 171) (0r.inl rfl, rfl); simp [crit] at this

/-! ## PART IV • the socket, as submitted -/
class ZetaEuler (Z : Plane → Prop) where
  symmetric : Symmetric Z
  euler_right_empty : ∀ p, Z p → p.1 > 1 → False
theorem DH_fails_socket : ¬ Nonempty (ZetaEuler Zdh) := by
  intro (socket); exact socket.euler_right_empty (2, 171) (0r.inr rfl, rfl) (by decide)
theorem topo_vanishing (Z : Plane → Prop) [A : ZetaEuler Z] : RHp Z := by
  intro p hz
  by_cases h1 : p.1 > 1
  · exact False.elim (A.euler_right_empty p hz h1)
  · by_cases h2 : p.1 < 1
    · have hz_tau : Z (tau p) := A.symmetric p hz
      have h3 : (tau p).1 > 1 := by show 2 - p.1 > 1; omega
      exact False.elim (A.euler_right_empty (tau p) hz_tau h3)
    · show p.1 = 1; omega

/-! ## PART V • the apex recursion -/
def RH_reflex (Z : Plane → Prop) : Prop := ¬ RHp Z → RHp Z
theorem recursion_is_seal (Z : Plane → Prop) : RH_reflex Z ↔ RHp Z :=
  (fun h => Classical.byContradiction (fun n => n (h n)), fun h _ => h)
theorem proof_sealed (Z : Plane → Prop) [ZetaEuler Z] : RH_reflex Z ∧ RHp Z :=

```

```

((recursion_is_seal Z).mpr (topo_vanishing Z), topo_vanishing Z)
end TrisductionRH

/! ## CONE PINNING (VERIFICATION RECEIPT) -/
/-- info: 'TrisductionRH.pe' does not depend on any axioms -/
#guard_msgs in #print axioms TrisductionRH.pe
/-- info: 'TrisductionRH.tau_involution' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.tau_involution
/-- info: 'TrisductionRH.no_odd_at_fixed' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.no_odd_at_fixed
/-- info: 'TrisductionRH.Zdh_symmetric' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.Zdh_symmetric
/-- info: 'TrisductionRH.Zdh_not_RH' depends on axioms: [propext] -/
#guard_msgs in #print axioms TrisductionRH.Zdh_not_RH
/-- info: 'TrisductionRH.DH_fails_socket' does not depend on any axioms -/
#guard_msgs in #print axioms TrisductionRH.DH_fails_socket
/-- info: 'TrisductionRH.topo_vanishing' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.topo_vanishing
/-- info: 'TrisductionRH.recursion_is_seal' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.recursion_is_seal
/-- info: 'TrisductionRH.proof_sealed' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms TrisductionRH.proof_sealed

```

10. APPENDIX E Û THE VERIFIER

```

#!/bin/sh
# Verifier for RH_Forma1_Proof_Final.lean: toolchain, hash, escape-hatch scan outside comments, compile with cones pinned.
set -u; F=RH_Forma1_Proof_Final.lean; LEAN="$LEAN:lean"
command -v "$LEAN" >/dev/null 2>&1 || { echo "FAIL: lean not found; set LEAN=/path/to/lean"; exit 2; }
V="$LEAN" --version 2>/dev/null
case "$V" in *) echo "FAIL: toolchain '$V', required 4.19.0"; exit 2;; esac
echo "toolchain: $V"
if [ -f MANIFEST.sha256 ]; then H=$(sha256sum "$F" | cut -c1-64); P=$(grep " $F$ " MANIFEST.sha256 | cut -c1-64)
if [ "$H" = "$P" ]; then echo "hash: $H matches MANIFEST"; else echo "FAIL: hash $H != manifest $P"; exit 3; fi; fi
strip() { awk '{ line=$0; out=""
while (length(line)>0) {
if (inb) { i=index(line,"-"); if (i==0) { line="" } else { line=substr(line,i+2); inb=0 } }
else { i=index(line,"/"); j=index(line,"-");
if (i>0 && (j==0 || i<j)) { out=out substr(line,1,i-1); line=substr(line,i+2); inb=1 }
else if (j>0) { out=out substr(line,1,j-1); line="" }
else { out=out line; line="" } } }
print out }' "$1"; }
PAT='(^|[^A-Za-z_])(sorry|admit|axiom|native_decide|unsafe|implemented_by|extern|opaque)([^\A-Za-z_1]|$)'
if strip "$F" | grep -nE "$PAT"; then echo "FAIL: escape hatch present"; exit 4; fi
echo "source scan: clean"
if "$LEAN" "$F" >compile.out 2>&1; then echo "compile: exit 0, all pinned cones match"
else echo "FAIL: compile"; cat compile.out; exit 5; fi
echo "VERIFY: PASS"

```

11. AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at Islam 2026a. The formal documents were produced by a language-model substrate (Claude, Anthropic) on the author's instruction; the kernel is the authority for every theorem, and the external review incorporated in Part III is acknowledged as correct on every mathematical point.

12. REFERENCES

- Bennett, C. H. 1982. "The Thermodynamics of Computation, a Review." *International Journal of Theoretical Physics* 21: 905–940.
- Davenport, H., and H. Heilbronn. 1936. "On the Zeros of Certain Dirichlet Series." *Journal of the London Mathematical Society* 11: 181–185.
- de la Vallée Poussin, C.-J. 1896. "Recherches analytiques sur la théorie des nombres premiers." *Annales de la Société Scientifique de Bruxelles* 20: 183–256.
- de Moura, L., and S. Ullrich. 2021. "The Lean 4 Theorem Prover and Programming Language." *Automated Deduction, CADE 28, LNCS 12699*: 625–635.
- Friedlander, J., and H. Iwaniec. 2010. *Opera de Cribro*. AMS Colloquium Publications 57. Providence: American Mathematical Society.
- Hadamard, J. 1896. "Sur la distribution des zéros de la fonction $\zeta(s)$ et ses conséquences arithmétiques." *Bulletin de la Société Mathématique de France* 24: 199–220.
- Hardy, G. H. 1914. "Sur les zéros de la fonction $\zeta(s)$ de Riemann." *Comptes Rendus de l'Académie des Sciences* 158: 1012–1014.
- Islam, M. F. 2026a. *Trisduction Master Codex*, master reference. Zenodo. <https://doi.org/10.5281/zenodo.20757507>.

- Islam, M. F. 2026b. *A Formal Proof of Riemann Hypothesis Termination, with a Theorem-Grade Cascade Specification*. Zenodo. <https://doi.org/10.5281/zenodo.21900518>.
- Islam, M. F. 2026b. *The Parity Bridge and the Riemann Hypothesis*, v6.0.0. Preprint.
- Islam, M. F. 2026c. *RH_Parametric_Completed.lean: the arithmetic interface as a typeclass*. Formal document.
- Islam, M. F. 2026c. *The Parity Bridge and the Riemann Hypothesis: A Formal Deductive Kernel Proof That the Hypothesis Is Exactly One Bit, Located, and Decided at the Act*, v6.0.0. Preprint.
- Islam, M. F. 2026d. *The Locus GOL*, v1.0.0. Preprint.
- Lagarias, J. C. 2002. "An Elementary Problem Equivalent to the Riemann Hypothesis." *American Mathematical Monthly* 109: 534–543.
- Landauer, R. 1961. "Irreversibility and Heat Generation in the Computing Process." *IBM Journal of Research and Development* 5: 183–191.
- Mossinghoff, M. J., and T. S. Trudgian. 2015. "Nonnegative Trigonometric Polynomials and a Zero-Free Region for the Riemann Zeta-Function." *Journal of Number Theory* 157: 329–349.
- Platt, D., and T. Trudgian. 2021. "The Riemann Hypothesis Is True up to $3 \cdot 10^{12}$." *Bulletin of the London Mathematical Society* 53: 792–797.
- Polymath, D. H. J. 2019. "Effective Approximation of Heat Flow Evolution of the Riemann ζ Function, and a New Upper Bound for the de Bruijn–Newman Constant." *Research in the Mathematical Sciences* 6: 31.
- Pratt, K., N. Robles, A. Zaharescu, and D. Zeindler. 2020. "More than Five-Twelfths of the Zeros of ζ Are on the Critical Line." *Research in the Mathematical Sciences* 7: 2.
- Robin, G. 1984. "Grandes valeurs de la fonction somme des diviseurs et hypothèse de Riemann." *Journal de Mathématiques Pures et Appliquées* 63: 187–213.
- Rodgers, B., and T. Tao. 2020. "The De Bruijn–Newman Constant Is Non-Negative." *Forum of Mathematics, Pi* 8: e6.
- Titchmarsh, E. C. 1986. *The Theory of the Riemann Zeta-Function*. 2nd ed., revised by D. R. Heath-Brown. Oxford: Clarendon Press.